

R Notebook - Airbnb Sevilla

1. Introducción y Objetivos

1.1. Contexto del Problema

El presente trabajo se centra en el análisis de datos del mercado de alquileres vacacionales en la ciudad de Sevilla, operado a través de la plataforma **Airbnb**.

¿Por qué es interesante y cuál es su impacto actual? El dominio de los alquileres turísticos es actualmente uno de los temas socioeconómicos más candentes a nivel local y nacional. Sevilla es uno de los principales focos turísticos de España, recibiendo a millones de visitantes anualmente. La irrupción de plataformas como Airbnb ha generado un volumen de negocio multimillonario, transformando por completo el mercado inmobiliario de la ciudad. Sin embargo, este fenómeno también ha traído consigo el debate sobre la turistificación, la gentrificación de barrios históricos y la subida del precio del alquiler residencial.

Entender este dominio mediante la Ciencia de Datos no solo es interesante desde una perspectiva de **inteligencia de negocio** (para inversores que buscan rentabilidad), sino también desde la **gestión pública** (para reguladores urbanísticos que necesitan comprender la saturación del Casco Antiguo).

1.2. Objetivos del Estudio

El objetivo principal de este proyecto es aplicar técnicas de *Machine Learning* y Análisis Exploratorio Avanzado para extraer *insights* de valor sobre la dinámica de precios y la profesionalización del sector. Para ello, planteamos tres preguntas de negocio fundamentales que cualquier inversor o analista se haría al estudiar este mercado. El dataset seleccionado es perfectamente representativo para responderlas, ya que contiene tanto la oferta de precios reales como la demanda latente (medida a través de las *reviews*).

Las preguntas a resolver son:

- 1. Segmentación del mercado: ¿Podemos identificar clústeres de alojamientos que se comporten de forma distinta al estándar del barrio?**
 - *Utilidad real:* Permite a inversores identificar nichos de mercado (ej. “Tiburones inmobiliarios” de alta rotación frente a “Habitaciones compartidas de bajo coste”) y entender qué modelo de negocio domina cada zona de Sevilla, más allá de la media administrativa.
- 2. Determinantes del Precio: ¿Qué variables determinan el precio de mercado en Sevilla y qué pisos están sobrevalorados?**
 - *Utilidad real:* Mediante modelos predictivos (Regresión), un inversor inmobiliario puede tasar objetivamente una vivienda basándose en sus características físicas (baños, capacidad) y detectar oportunidades de inversión (pisos infravalorados) frente a trampas de mercado (pisos con precio inflado sin justificación).
- 3. El “Efecto Giralda”: ¿Cómo cae el precio del alquiler turístico por cada 100 metros que nos alejamos de la Catedral?**
 - *Utilidad real:* Convierte la geografía en matemáticas. Al cuantificar el decaimiento exacto del precio por metro de alejamiento, una agencia inmobiliaria puede calcular si el sobrecoste de comprar un piso en pleno centro se amortiza con la prima de facturación turística diaria.

1.3. Descripción del Dataset

Para abordar estas cuestiones, hemos utilizado un conjunto de datos público extraído del portal **Inside Airbnb**(<https://insideairbnb.com/get-the-data/>), un proyecto independiente que proporciona datos scrapeados directamente de la plataforma oficial para el análisis de su impacto en ciudades de todo el mundo.

- **Origen de los datos:** Archivo `listings.csv` correspondiente a la ciudad de Sevilla.
- **Volumen de observaciones (Filas):** El dataset original cuenta con **8.215 observaciones** (cada fila representa un alojamiento único ofertado en la ciudad).
- **Volumen de características (Columns):** Contiene **79 variables** originales que detallan desde las coordenadas geográficas hasta las políticas de reserva y calificaciones de los huéspedes.
- **Tamaño del archivo:** El dataset en crudo ocupa aproximadamente **18.5 MB** de memoria física, lo que justifica la necesidad de aplicar técnicas de preprocesamiento, limpieza y reducción de dimensionalidad para optimizar el rendimiento computacional de nuestros algoritmos.

2. Selección de variables y preprocesamiento de datos

El dataset original de Airbnb (`listings.csv`) cuenta con **79 variables** y más de 8.000 observaciones. Gran parte de esta información consiste en metadatos técnicos (`scrape_id`), textos no estructurados (`description`, `urls`) o redundancias que introducen ruido en nuestro análisis.

Para abordar las tres preguntas de negocio planteadas (Segmentación por barrio, Predicción de Precio y Análisis Geoespacial), hemos ejecutado una **selección estratégica de características (Feature Selection)**. Hemos reducido la dimensionalidad del dataset a **20 variables clave**, agrupadas en cuatro dimensiones funcionales que explican la varianza del mercado:

A. Dimensiones Estructurales (“El Hardware”) Variables necesarias para estandarizar la comparación entre alojamientos heterogéneos.

- **room_type, accommodates, bedrooms:** Definen la capacidad real y el modelo de alojamiento (privado vs. compartido). Son los predictores base del precio.
- **bathrooms_text:** Seleccionada para ingeniería de características. El número de baños es un indicador crítico de lujo/gama alta que a menudo se omite en análisis básicos.
- **amenities:** Aunque es texto, la transformaremos para cuantificar el “valor añadido” (piscina, aire acondicionado) que justifica sobreprecios.

B. Dimensiones Económicas y de Gestión (“El Negocio”) Variables que permiten perfilar el comportamiento del anfitrión (Profesional vs. Particular).

- **price:** Variable objetivo principal. Requiere limpieza de caracteres (\$, ,) para su tratamiento numérico.
- **availability_365:** Proxy de dedicación. Distingue entre negocios de dedicación exclusiva y alquileres esporádicos.
- **calculated_host_listings_count** e **instant_bookable:** Métricas de profesionalización. Un anfitrión con múltiples propiedades y reserva inmediata tiene un perfil de riesgo y rotación distinto.

C. Dimensiones de Reputación y Calidad (“El Valor Percibido”) Cruciales para el Clustering y para explicar outliers de precio.

- **reviews_per_month:** El mejor indicador de la demanda actual y la rotación del activo.

- **review_scores_rating, _cleanliness, _location:** Variables de calidad subjetiva. Permiten detectar alojamientos “sobrevalorados” (caros pero con mala nota) o “joyas ocultas”.

D. Dimensiones Geoespaciales (“La Ubicación”) Necesarias para el análisis continuo del espacio urbano.

- **latitude / longitude:** Materia prima para el cálculo de distancia euclídea/haversine a la Giralda (Pregunta 3).
- **neighbourhood_cleansed:** Para establecer la línea base de precios por zona administrativa (Pregunta 2).

```
library(tidyverse)
library(corrplot)
```

```
listings <- read.csv("./csv/listings.csv")
```

```
str(listings)
```

```
## 'data.frame':    8215 obs. of  79 variables:
```

##	\$ id	: num	49287 108236 111140 159596 179629 ...
##	\$ listing_url	: chr	"https://www.airbnb.com/rooms/49287" "https://"
##	\$ scrape_id	: num	2.03e+13 2.03e+13 2.03e+13 2.03e+13 2.03e+13 .
##	\$ last_scraped	: chr	"2025-09-30" "2025-09-30" "2025-09-30" "2025-09-
##	\$ source	: chr	"city scrape" "city scrape" "city scrape" "prev
##	\$ name	: chr	"BEAUTIFUL APARTMENT IN SEVILLE" "Sunny apt in
##	\$ description	: chr	"Nice apartment on the second floor of a beaut.
##	\$ neighborhood_overview	: chr	"The famous Plaza de Hercules has changed a lo
##	\$ picture_url	: chr	"https://a0.muscache.com/pictures/7a7b22d8-5f7
##	\$ host_id	: int	224697 560040 560040 629861 860055 860055 1022
##	\$ host_url	: chr	"https://www.airbnb.com/users/show/224697" "ht
##	\$ host_name	: chr	"Walter" "José Luis" "José Luis" "Alvaro" ...
##	\$ host_since	: chr	"2010-09-05" "2011-05-05" "2011-05-05" "2011-0
##	\$ host_location	: chr	"Sevilla, Spain" "Seville, Spain" "Seville, Sp
##	\$ host_about	: chr	"Born in Holland and moved to Sevilla in 1997\
##	\$ host_response_time	: chr	"within an hour" "within an hour" "within an h
##	\$ host_response_rate	: chr	"100%" "100%" "100%" "0%" ...
##	\$ host_acceptance_rate	: chr	"100%" "92%" "92%" "0%" ...
##	\$ host_is_superhost	: chr	"f" "t" "t" "f" ...
##	\$ host_thumbnail_url	: chr	"https://a0.muscache.com/im/users/224697/profi
##	\$ host_picture_url	: chr	"https://a0.muscache.com/im/users/224697/profi
##	\$ host_neighbourhood	: chr	"Casco Antiguo" "Casco Antiguo" "Casco Antiguo
##	\$ host_listings_count	: int	1 4 4 2 24 24 8 2 2 7 ...
##	\$ host_total_listings_count	: int	1 10 10 2 39 39 8 4 2 18 ...
##	\$ host_verifications	: chr	"['email', 'phone', 'work_email']" '['email',
##	\$ host_has_profile_pic	: chr	"t" "t" "t" "t" ...
##	\$ host_identity_verified	: chr	"t" "t" "t" "t" ...
##	\$ neighbourhood	: chr	"Seville, AL, Spain" "Seville, Andalusia, Spain
##	\$ neighbourhood_cleansed	: chr	"San Lorenzo" "San Vicente" "San Vicente" "Aren
##	\$ neighbourhood_group_cleansed	: chr	"Casco Antiguo" "Casco Antiguo" "Casco Antiguo
##	\$ latitude	: num	37.4 37.4 37.4 37.4 37.4 ...
##	\$ longitude	: num	-6 -6 -6 -6 -5.99 ...
##	\$ property_type	: chr	"Entire rental unit" "Entire rental unit" "Ent
##	\$ room_type	: chr	"Entire home/apt" "Entire home/apt" "Entire hom
##	\$ accommodates	: int	3 5 4 4 5 7 2 1 2 6 ...
##	\$ bathrooms	: num	1 1 1 NA 1 NA 11.5 1 1 ...

```

## $ bathrooms_text           : chr "1 bath" "1 bath" "1 bath" "1 bath" ...
## $ bedrooms                 : int 1 2 1 2 3 3 1 1 1 2 ...
## $ beds                     : int 1 3 2 NA 4 NA 1 1 1 4 ...
## $ amenities                 : chr "[\"Bed linens\", \"Air conditioning\", \"Stove\" ...
## $ price                     : chr \"$93.00\" \"$120.00\" \"$89.00\" \"\" ...
## $ minimum_nights           : int 3 2 2 64 3 3 3 1 3 2 ...
## $ maximum_nights           : int 1125 1125 1125 330 1125 365 30 7 365 365 ...
## $ minimum_minimum_nights   : int 3 2 2 64 2 3 2 1 3 2 ...
## $ maximum_minimum_nights   : int 3 2 2 64 3 3 3 3 3 3 ...
## $ minimum_maximum_nights   : int 1125 1125 1125 330 1125 20 1125 7 365 365 ...
## $ maximum_maximum_nights   : int 1125 1125 1125 330 1125 1125 1125 7 365 365 ...
## $ minimum_nights_avg_ntm   : num 3 2 2 64 3 3 3 1.1 3 2 ...
## $ maximum_nights_avg_ntm   : num 1125 1125 1125 330 1125 ...
## $ calendar_updated         : logi NA NA NA NA NA NA ...
## $ has_availability          : chr "t" "t" "t" "t" ...
## $ availability_30           : int 5 5 2 0 7 8 8 3 11 4 ...
## $ availability_60           : int 18 35 19 30 22 32 29 32 38 30 ...
## $ availability_90           : int 41 65 38 60 37 45 56 62 63 32 ...
## $ availability_365          : int 206 79 106 60 245 252 111 62 148 32 ...
## $ calendar_last_scraped     : chr "2025-09-30" "2025-09-30" "2025-09-30" "2025-09-30" ...
## $ number_of_reviews         : int 40 222 63 1 205 120 508 238 331 459 ...
## $ number_of_reviews_ltm     : int 1 25 11 0 3 7 53 41 18 34 ...
## $ number_of_reviews_l30d    : int 1 3 0 0 0 0 6 1 1 2 ...
## $ availability_eoy          : int 41 68 38 60 37 46 59 62 64 32 ...
## $ number_of_reviews_ly      : int 0 16 10 0 13 10 56 32 35 38 ...
## $ estimated_occupancy_l365d : int 6 150 66 0 18 42 255 246 108 204 ...
## $ estimated_revenue_l365d   : int 558 18000 5874 NA 2628 NA 21675 13284 8640 214 ...
## $ first_review              : chr "2011-02-27" "2011-05-17" "2011-05-27" "2019-1 ...
## $ last_review               : chr "2025-09-21" "2025-09-23" "2025-08-02" "2019-1 ...
## $ review_scores_rating      : num 4.64 4.77 4.7 4 4.55 4.4 4.87 4.94 4.78 4.61 ...
## $ review_scores_accuracy    : num 4.87 4.84 4.68 4 4.59 4.6 4.93 4.92 4.85 4.72 ...
## $ review_scores_cleanliness : num 4.97 4.81 4.69 5 4.77 4.73 4.95 4.96 4.68 4.72 ...
## $ review_scores_checkin     : num 4.92 4.88 4.87 5 4.5 4.48 4.93 4.89 4.89 4.72 ...
## $ review_scores_communicaion : num 4.92 4.93 4.84 5 4.55 4.59 4.96 4.91 4.92 4.66 ...
## $ review_scores_location    : num 4.92 4.86 4.82 5 4.93 4.17 4.94 4.91 4.93 4.88 ...
## $ review_scores_value       : num 4.74 4.75 4.52 4 4.6 4.54 4.8 4.86 4.77 4.63 ...
## $ license                   : chr "VFT/SE/01116" "VFT/SE/05126" "VUT/SE/08197" " ...
## $ instant_bookable          : chr "t" "f" "f" "f" ...
## $ calculated_host_listings_count : int 1 4 4 2 24 24 1 2 2 5 ...
## $ calculated_host_listings_count_entire_homes : int 1 4 4 2 24 24 1 0 2 5 ...
## $ calculated_host_listings_count_private_rooms : int 0 0 0 0 0 0 0 2 0 0 ...
## $ calculated_host_listings_count_shared_rooms : int 0 0 0 0 0 0 0 0 0 0 ...
## $ reviews_per_month         : num 0.23 1.27 0.36 0.01 1.23 0.7 3.78 1.41 1.96 2. ...

```

Antes de proceder al preprocesamiento, realizamos una inspección técnica del dataset crudo. De esta forma podemos analizar aspectos claves como:

- Duplicidad
- Missing values
- Consistencia

```
# 1. ESTRUCTURA
```

```
print("--- Dimensiones del Dataset ---")
```

```
## [1] "--- Dimensiones del Dataset ---"
```

```

dim(listings) # Filas x Columnas

## [1] 8215    79

# 2. CHECK DE DUPLICADOS
duplicados_totales <- sum(duplicated(listings))
duplicados_id <- sum(duplicated(listings$id))

print(paste("Filas totalmente duplicadas:", duplicados_totales))

## [1] "Filas totalmente duplicadas: 0"

print(paste("IDs repetidos (mismo piso scrapeado varias veces):", duplicados_id))

## [1] "IDs repetidos (mismo piso scrapeado varias veces): 0"

# 3. MISSING VALUES
na_count <- colSums(is.na(listings))
na_percent <- (na_count / nrow(listings)) * 100

# Mostrar las 10 columnas con más Nulos
print("--- Top 10 Columnas con más Nulos (%) ---")

## [1] "--- Top 10 Columnas con más Nulos (%) ---"

print(sort(na_percent, decreasing = TRUE)[1:10])

##          calendar_updated      review_scores_rating      review_scores_accuracy      review_scores_
##          100.00000          8.82532          8.82532          8.82532
##      review_scores_checkin review_scores_communication      review_scores_location      review_s
##          8.82532          8.82532          8.82532          8.82532
##      reviews_per_month      estimated_revenue_l365d
##          8.82532          7.71759

# 4. Variables clave Room Type y Precios
# Ver si hay categorías raras antes de limpiar
print("--- Categorías únicas en Room Type ---")

## [1] "--- Categorías únicas en Room Type ---"

print(table(listings$room_type))

##
## Entire home/apt      Hotel room      Private room      Shared room
##          6993          21          1182          19

# Ver formato del precio por si hay que limpiar
print("--- Ejemplo de Precios en crudo ---")

## [1] "--- Ejemplo de Precios en crudo ---"

print(head(listings$price))

## [1] "$93.00" "$120.00" "$89.00" "" "$146.00" ""

```

2.1. EDA inicial

En este punto hemos realizado un EDA pequeño, para hacernos una idea de donde partíamos, con qué tipo de datos tratamos, formato, valores nulos, incongruencias, en definitiva, qué sobra y qué nos parece interesante tratar y preprocesar en detalle.

```
library(ggplot2)
library(dplyr)

# GRÁFICO 1: EL "MONSTRUO" DE LOS PRECIOS
# Necesitamos convertir el precio a número "en sucio" solo para poder pintarlo
# (Sin modificar el dataset original todavía)
precios_sucios <- as.numeric(gsub("[\\$,]", "", listings$price))

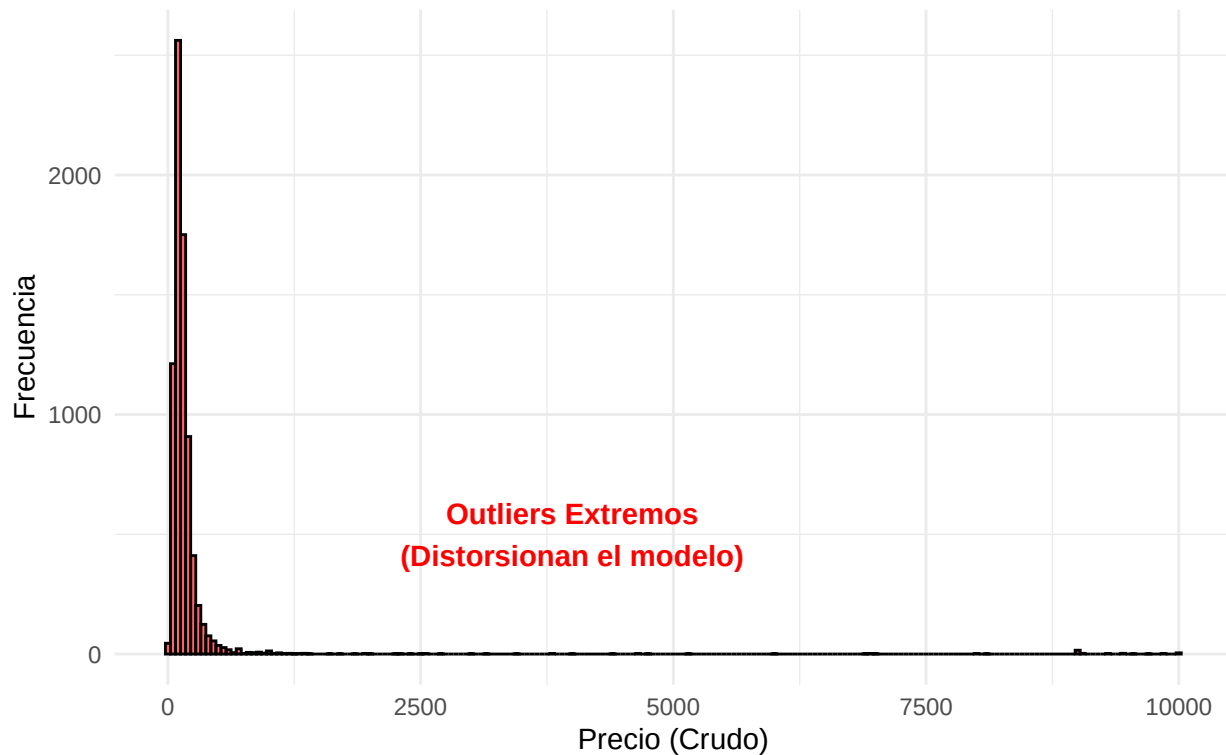
# Creamos un dataframe temporal solo para este gráfico
df_plot_sucio <- data.frame(precio = precios_sucios)

ggplot(df_plot_sucio, aes(x = precio)) +
  geom_histogram(binwidth = 50, fill = "red", color = "black", alpha = 0.6) +
  theme_minimal() +
  labs(
    title = "Distribución Original de Precios (Sin Limpiar)",
    subtitle = "Extrema asimetría debido a outliers (pisos de 5.000€+)",
    x = "Precio (Crudo)",
    y = "Frecuencia"
  ) +
  annotate("text", x = 4000, y = 500, label = "Outliers Extremos\n(Distorsionan el modelo)", color = "red")
```

Warning: Removed 634 rows containing non-finite outside the scale range (`stat\_bin()`).

Distribución Original de Precios (Sin Limpiar)

Extrema asimetría debido a outliers (pisos de 5.000€+)



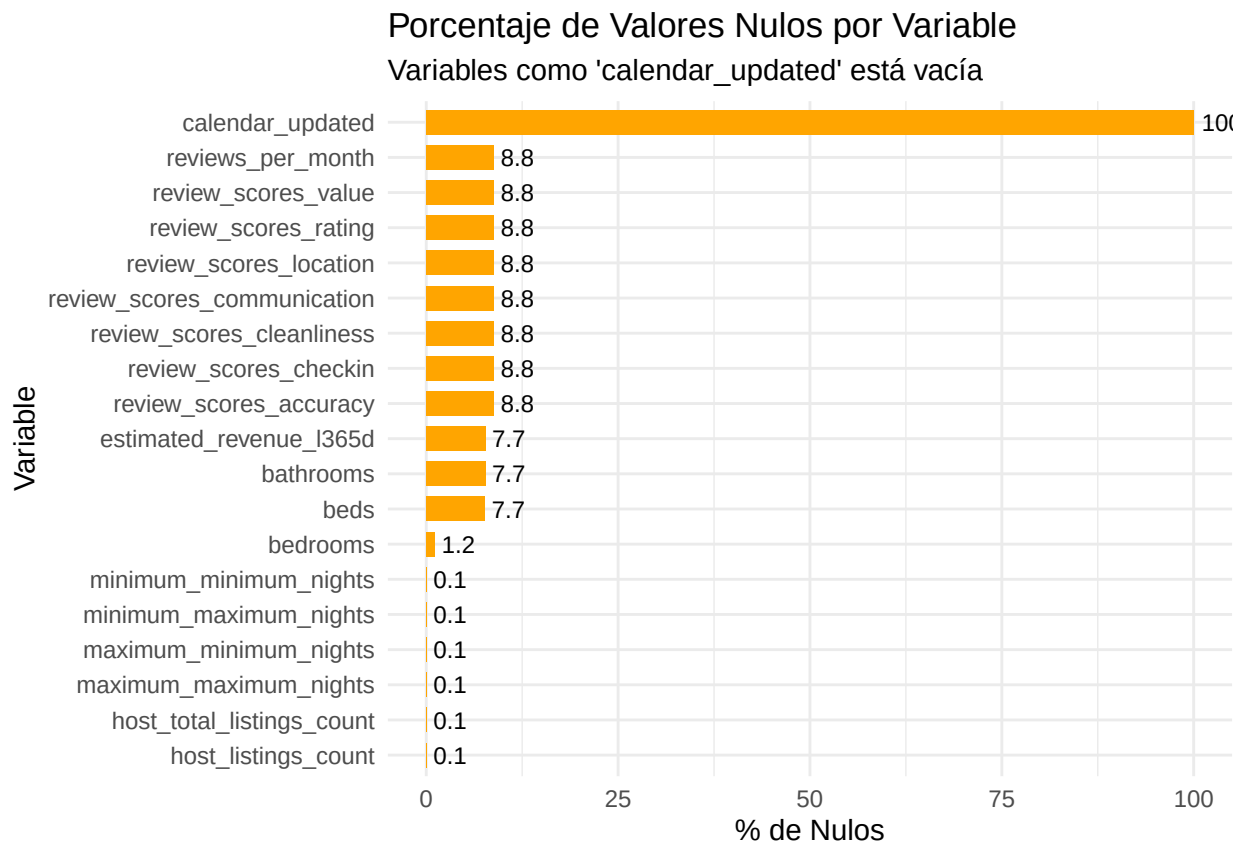
```
# GRÁFICO 2: MAPA DE CALOR DE DATOS FALTANTES (NULOS)
# Calculamos el % de nulos por columna
na_summary <- data.frame(
```

```

columna = names(listings),
na_pct = colSums(is.na(listings)) / nrow(listings) * 100
) %>%
filter(na_pct > 0) %>% # Solo mostramos las que tienen nulos
arrange(desc(na_pct)) # Ordenamos de mayor a menor

# Pintamos solo las columnas problemáticas
ggplot(na_summary, aes(x = reorder(columna, na_pct), y = na_pct)) +
  geom_bar(stat = "identity", fill = "orange", width = 0.7) +
  coord_flip() + # Giramos para leer los nombres
  theme_minimal() +
  labs(
    title = "Porcentaje de Valores Nulos por Variable",
    subtitle = "Variables como 'calendar_updated' está vacía",
    x = "Variable",
    y = "% de Nulos"
  ) +
  geom_text(aes(label = round(na_pct, 1)), hjust = -0.2, size = 3)

```



Tras este pequeño análisis decidimos seleccionar las columnas que nos impactan. Así, podemos centrarnos en analizar profundamente solo las variables necesarias, y hacer el preprocesado detallado y centrado en nuestro caso concreto y no extendernos en algo que nos dará fruto para resolver nuestra hipótesis planteadas.

```

# 1. SELECCIÓN DE COLUMNAS
df_raw_selected <- listings %>%
  select(
    # Identificador

```

```

id,

# Variables clave (Precios, Tipo y Estancia)
price,
room_type,
accommodates,
minimum_nights,

# Variables estructurales
bathrooms_text,
bedrooms,
amenities,

# Ubicación
neighbourhood_cleansed,
neighbourhood_group_cleansed,
latitude,
longitude,

# Actividad y reputación
reviews_per_month,
number_of_reviews,
review_scores_rating,
review_scores_cleanliness,
review_scores_location,

# Gestión
availability_365,
calculated_host_listings_count,
instant_bookable
)

print(paste("Columnas seleccionadas:", ncol(df_raw_selected)))

## [1] "Columnas seleccionadas: 20"

```

2.2. Preprocesado de Variables Críticas

```

df_final <- df_raw_selected %>%

# --- 1. LIMPIEZA DE PRECIO ---
mutate(
  price_clean = as.numeric(gsub("[\\$,]", "", price))
) %>%
# Filtrar precios coherentes
filter(price_clean > 10 & price_clean < 1000) %>%

# --- 2. LIMPIEZA DE BAÑOS (VERSIÓN MEJORADA) ---
mutate(
  # 2.1. Sacamos el NÚMERO (Cantidad)
  bathrooms_qty = as.numeric(str_extract(bathrooms_text, "\\d+(\\.\\d+)?")),
  bathrooms_qty = ifelse(is.na(bathrooms_qty) & str_detect(bathrooms_text, "Half-bath"), 0.5, bathrooms_qty),
  bathrooms_qty = ifelse(is.na(bathrooms_qty), 1, bathrooms_qty),

```



```

# 2.2. Sacamos la PRIVACIDAD (Calidad)
# Si detecta "shared", pone un 1. Si no, un 0.
bath_shared = ifelse(str_detect(bathrooms_text, regex("shared", ignore_case = TRUE)), 1, 0)
) %>%

# --- 3. INGENIERÍA DE AMENITIES ---
mutate(
  amenities_count = str_count(amenities, ",") + 1,
  amenities_count = ifelse(is.na(amenities) | amenities == "" | amenities == "[]", 0, amenities_count),
  has_pool = as.integer(str_detect(amenities, regex("pool", ignore_case = TRUE))),
  has_ac = as.integer(str_detect(amenities, regex("air conditioning|ac", ignore_case = TRUE)))
) %>%

# --- 4. TRATAMIENTO DE NULOS ---
group_by(room_type)%>%
  mutate(
    reviews_per_month = replace_na(reviews_per_month, 0),
    bedrooms = ifelse(is.na(bedrooms), median(bedrooms, na.rm = TRUE), bedrooms),
    review_scores_rating = ifelse(is.na(review_scores_rating), mean(review_scores_rating, na.rm = TRUE), review_scores_rating),
    review_scores_cleanliness = ifelse(is.na(review_scores_cleanliness), mean(review_scores_cleanliness, na.rm = TRUE), review_scores_cleanliness),
    review_scores_location = ifelse(is.na(review_scores_location), mean(review_scores_location, na.rm = TRUE), review_scores_location),
    instant_bookable_binary = ifelse(instant_bookable == "t", 1, 0),

    # LIMPIEZA DE MINIMUM_NIGHTS
    minimum_nights = ifelse(is.na(minimum_nights), 1, minimum_nights)
  ) %>%
ungroup() %>%
# --- 5. SELECCIÓN FINAL ---
select(
  id,
  price = price_clean,
  room_type,
  accommodates,
  minimum_nights,
  bedrooms,
  bathrooms = bathrooms_qty,
  bath_shared,
  amenities_count, has_pool, has_ac,
  neighbourhood_cleansed,
  neighbourhood_group_cleansed,
  latitude, longitude,
  reviews_per_month, number_of_reviews,
  review_scores_rating, review_scores_cleanliness, review_scores_location,
  availability_365, calculated_host_listings_count,
  instant_bookable = instant_bookable_binary
) %>%

# --- 6. CONVERSIÓN A FACTORES ---
# Convertimos texto y binarios a categorías para que R los entienda bien en los gráficos
mutate(
  room_type = as.factor(room_type),
  neighbourhood_cleansed = as.factor(neighbourhood_cleansed)
)

```

```
# Verificar estructura final
glimpse(df_final)
```

```
## Rows: 7,484
## Columns: 23
## $ id <dbl> 49287, 108236, 111140, 179629, 207702, 227905, 253430, 266016
## $ price <dbl> 93, 120, 89, 146, 85, 54, 80, 105, 127, 49, 103, 104, 132, 63
## $ room_type <fct> Entire home/apt, Entire home/apt, Entire home/apt, Entire hom
## $ accommodates <int> 3, 5, 4, 5, 2, 1, 2, 6, 2, 1, 4, 2, 4, 2, 4, 4, 5, 2, 10, .
## $ minimum_nights <int> 3, 2, 2, 3, 3, 1, 3, 2, 2, 3, 1, 3, 2, 2, 1, 3, 3, 2, 3, 2, 3
## $ bedrooms <dbl> 1, 2, 1, 3, 1, 1, 1, 2, 1, 1, 2, 1, 2, 1, 1, 1, 2, 3, 0, 3, 1
## $ bathrooms <dbl> 1.0, 1.0, 1.0, 1.0, 1.0, 1.5, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1
## $ bath_shared <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
## $ amenities_count <dbl> 50, 63, 53, 29, 37, 25, 35, 28, 51, 39, 33, 46, 42, 20, 33, 3
## $ has_pool <int> 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
## $ has_ac <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
## $ neighbourhood_cleansed <fct> "San Lorenzo", "San Vicente", "San Vicente", "Santa Cruz", "M
## $ neighbourhood_group_cleansed <chr> "Casco Antiguo", "Casco Antiguo", "Casco Antiguo", "Casco Ant
## $ latitude <dbl> 37.39898, 37.39686, 37.39592, 37.38731, 37.38945, 37.39299, 3
## $ longitude <dbl> -5.995330, -5.999127, -5.999317, -5.990950, -5.998890, -5.999
## $ reviews_per_month <dbl> 0.23, 1.27, 0.36, 1.23, 3.78, 1.41, 1.96, 2.76, 3.71, 3.14, 2
## $ number_of_reviews <int> 40, 222, 63, 205, 508, 238, 331, 459, 611, 519, 474, 686, 58,
## $ review_scores_rating <dbl> 4.64, 4.77, 4.70, 4.55, 4.87, 4.94, 4.78, 4.61, 4.87, 4.75, 4
## $ review_scores_cleanliness <dbl> 4.97, 4.81, 4.69, 4.77, 4.95, 4.96, 4.68, 4.72, 4.91, 4.90, 4
## $ review_scores_location <dbl> 4.92, 4.86, 4.82, 4.93, 4.94, 4.91, 4.93, 4.88, 4.96, 4.84, 4
## $ availability_365 <int> 206, 79, 106, 245, 111, 62, 148, 32, 120, 60, 68, 227, 60, 19
## $ calculated_host_listings_count <int> 1, 4, 4, 24, 1, 2, 2, 5, 205, 2, 1, 1, 6, 2, 4, 1, 1, 1, 4, 9
## $ instant_bookable <dbl> 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1
```

```
summary(df_final$minimum_nights)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.     Max.
##      1.00    1.00    2.00    2.96    2.00   360.00
```

Para garantizar la robustez de los modelos posteriores (Clustering y Regresión), hemos implementado una transformación sobre el dataset original (`df_raw_selected`). Las operaciones realizadas son las siguientes:

1. Limpieza y Filtrado de la Variable Objetivo (Price) La variable precio original contenía caracteres no numéricos (\$ y ,). Se ha convertido a tipo numérico y posterior filtrado.

- **Decisión:** Se han eliminado registros con precios inferiores a **10€** (errores de carga o precios ínfimos) y superiores a **1.000€** (outliers extremos/mansiones).

Esto reduce la asimetría de la distribución y evita que los valores extremos distorsionen la media y los centroides en el análisis de K-Means.

2. Extracción de Información Estructural (Bathrooms) La columna `bathrooms_text` presentaba información no estructurada (ej. “1.5 baths”). Hemos realizado las siguientes acciones:

- Extraído el valor numérico.
- Hemos imputado el valor **0.5** para los casos etiquetados como “Half-bath”.
- Hemos asumido un valor estándar de **1** baño para los valores nulos restantes.

3. Características de Servicios (Amenities) La columna `amenities` es una lista de texto JSON compleja, muy interesante porque podemos recoger qué características “deluxe” tienen los pisos. Para ello, hemos creado tres nuevas variables sintéticas y así cuantificar el valor añadido:

- **amenities_count**: Conteo del número total de servicios ofrecidos, como indicador general del nivel de equipamiento.
- **has_pool** y **has_ac**: Variables binarias (1/0) creadas mediante búsqueda de patrones de texto. Consideramos que en un sitio como Sevilla, la piscina y el aire acondicionado son críticos para el precio (Pregunta 2).

4. Estrategia seguida para la Imputación de Valores Nulos Para no perder observaciones valiosas, hemos aplicado una estrategia de imputación diferente, según cada variable:

- **Actividad (reviews_per_month)**: Los nulos se imputan con **0**, interpretando la ausencia de dato como actividad nula.
- **Estructural (bedrooms)**: Se imputa con la **mediana condicionada** por la tipología del alojamiento **room_type**, en lugar de emplear la mediana global. Esto nos permite preservar la heterogeneidad y la varianza intrínseca de cada modelo de alojamiento, evitando sesgos.
- **Calidad (review_scores)**: Se imputa con la **media** global para mantener la neutralidad de la observación sin penalizarla artificialmente.
- **Noches mínimas (minimum_nights)**: Se imputan los nulos con **1**, asumiendo la estancia mínima estándar.

5. Selección y Renombrado Final : hemos generado el dataset **df_final** descartando las variables intermedias sucias (como **bathrooms_text** o el **price** original) y reteniendo únicamente las **19 variables limpias y transformadas** que necesitaremos para trabajar los tres modelos del proyecto.

```
head(df_final)
```

```
## # A tibble: 6 x 23
##       id price room_type      accommodates minimum_nights bedrooms bathrooms bath_shared amenities_
##   <dbl> <dbl> <fct>          <int>          <int>      <dbl>      <dbl>      <dbl>      <dbl>
## 1  49287   93 Entire home/apt           3           3         1         1         0
## 2 108236  120 Entire home/apt           5           2         2         1         0
## 3 111140   89 Entire home/apt           4           2         1         1         0
## 4 179629  146 Entire home/apt           5           3         3         1         0
## 5 207702   85 Entire home/apt           2           3         1         1         0
## 6 227905   54 Private room             1           1         1         1.5       1
## # i 12 more variables: neighbourhood_cleansed <fct>, neighbourhood_group_cleansed <chr>, latitude <dbl>,
## #   reviews_per_month <dbl>, number_of_reviews <int>, review_scores_rating <dbl>, review_scores_cleanliness
## #   review_scores_location <dbl>, availability_365 <int>, calculated_host_listings_count <int>, instant_bookable <dbl>

# Asumiendo que ya cargaste library(tidyverse) o library(readr)
write_csv(df_final, file = "./csv/listings_preprocesado.csv")
```

2.3. Filtrado de Inconsistencias

```
# --- 2.3. FILTRADO DE INCONSISTENCIAS Y OUTLIERS ---
# Partimos de df_final que ya tiene las variables limpias (baños, amenities, etc.)

df_eda <- df_final %>%
  # A. Filtro de Precio: Ajustamos a tu rango (10€ - 1.000€)
  filter(price > 10 & price < 1000) %>%

  # B. Filtro Geoespacial: Seguridad por si falló algo
  filter(!is.na(latitude) & !is.na(longitude)) %>%

  # C. Filtro de Estancias: Eliminamos alquileres de año completo (posibles errores)
```

```

filter(minimum_nights < 365)

# Ver cuántas filas nos quedan vs las originales
print(paste("Observaciones originales:", nrow(df_final)))

## [1] "Observaciones originales: 7484"

print(paste("Observaciones para análisis (df_eda):", nrow(df_eda)))

## [1] "Observaciones para análisis (df_eda): 7484"

```

3. Análisis Exploratorio de Datos

3.1. Análisis Univariante

3.1.1 Distribución del precio

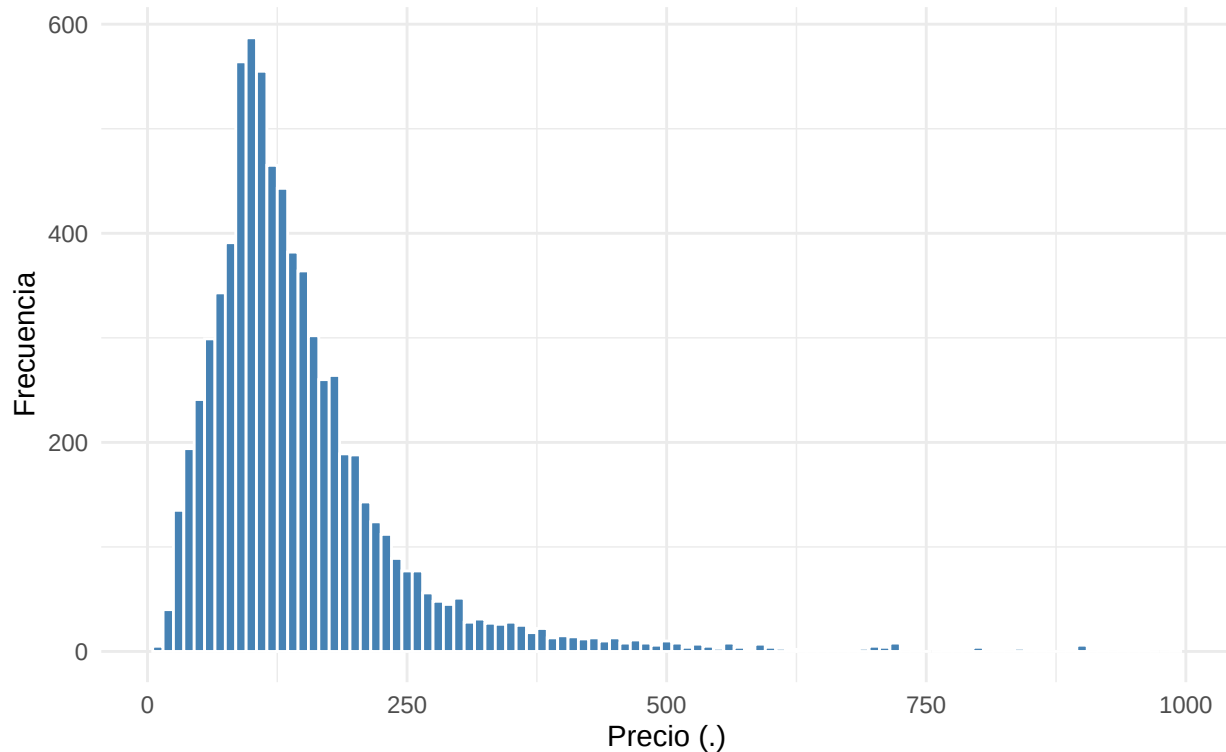
```

# 3.1.1 Distribución del Precio
ggplot(df_eda, aes(x = price)) +
  geom_histogram(binwidth = 10, fill = "steelblue", color = "white") +
  theme_minimal() +
  labs(title = "Distribución de Precios (Filtrado 10€ - 1000€)",
       subtitle = "La mayoría del mercado se concentra bajo los 200€",
       x = "Precio (€)", y = "Frecuencia")

```

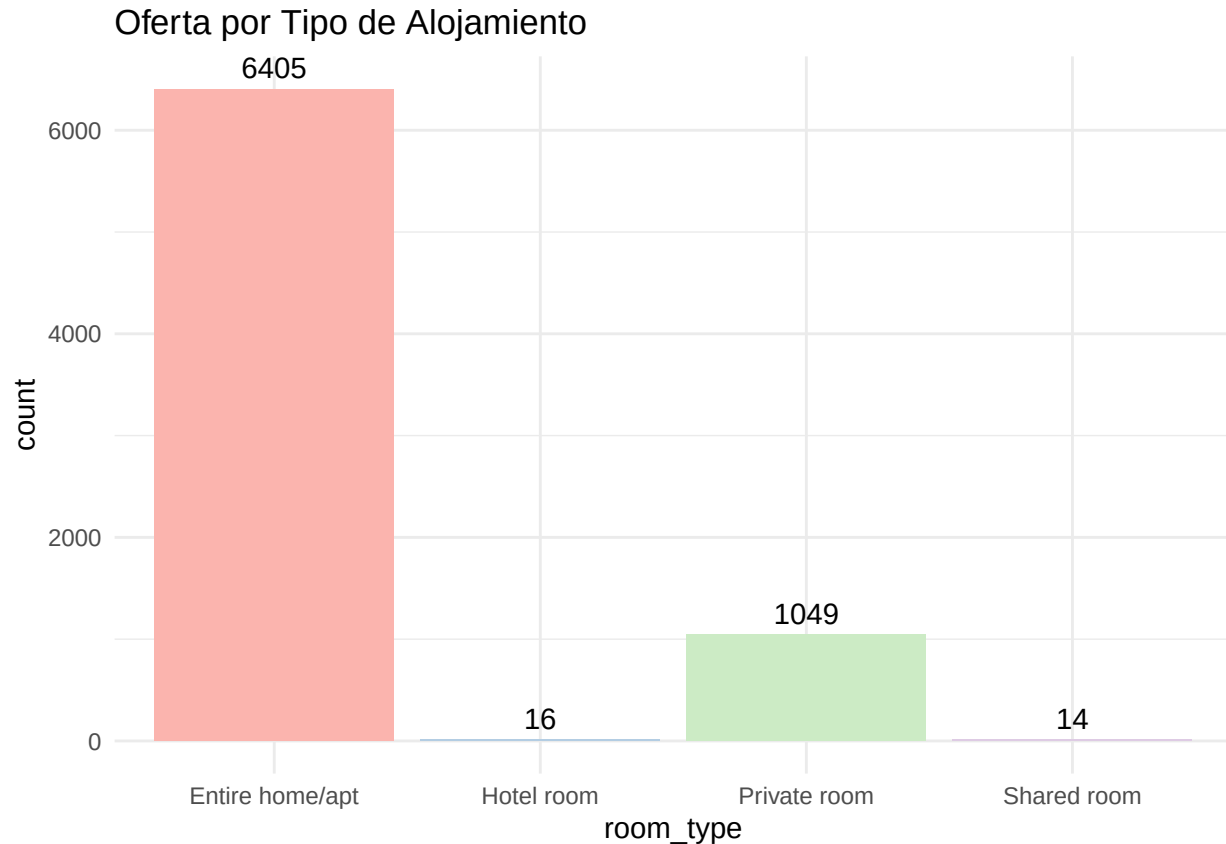
Distribución de Precios (Filtrado 10. – 1000.)

La mayoría del mercado se concentra bajo los 200.



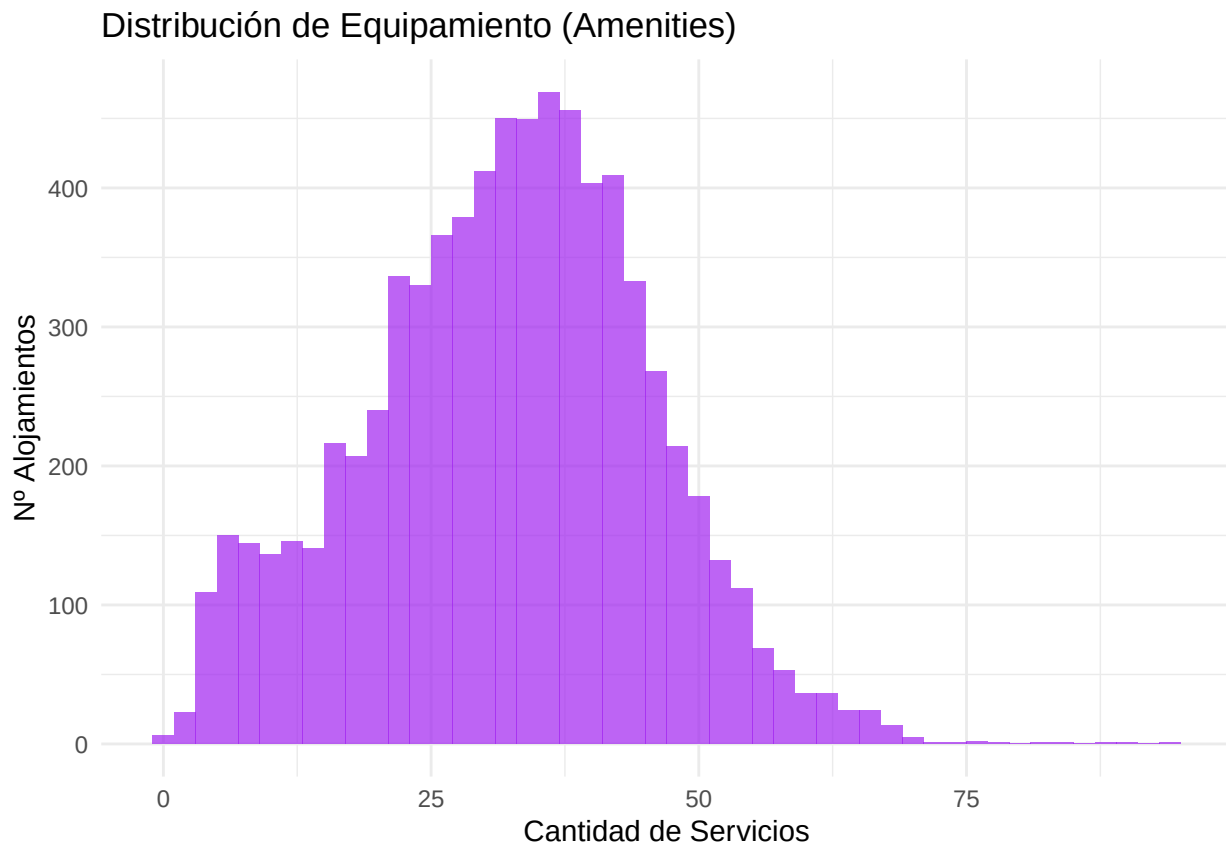
3.1.2 Conteo del tipo de habitacion

```
# 3.1.2 Conteo por Tipo de Habitación
ggplot(df_eda, aes(x = room_type, fill = room_type)) +
  geom_bar() +
  geom_text(stat='count', aes(label=..count..), vjust=-0.5) +
  theme_minimal() +
  scale_fill_brewer(palette = "Pastell1") +
  labs(title = "Oferta por Tipo de Alojamiento") +
  theme(legend.position = "none")
```



3.1.3 Boxplot de noches minimas

```
# 3.1.3 Distribución de Servicios (Amenities Count)
# Importante para justificar que hay pisos "pelados" y pisos "equipados"
ggplot(df_eda, aes(x = amenities_count)) +
  geom_histogram(binwidth = 2, fill = "purple", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Distribución de Equipamiento (Amenities)",
       x = "Cantidad de Servicios", y = "Nº Alojamientos")
```



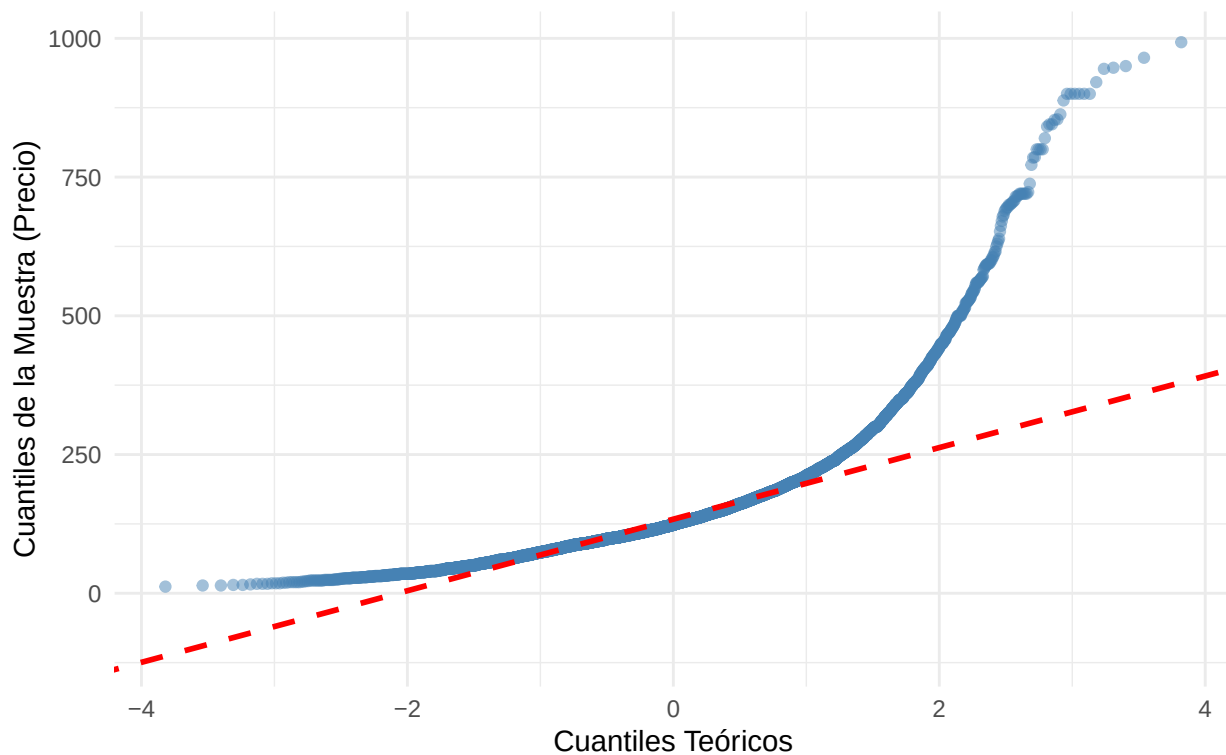
3.1.4 Validación de Normalidad del Precio

¿Por qué este gráfico? Siguiendo el Tema 5 (Análisis Univariante), usamos el QQ-Plot para comprobar si la variable objetivo (price) sigue una distribución normal. La desviación de los puntos respecto a la línea diagonal nos confirmará visualmente la fuerte asimetría positiva de los precios, justificando así posibles transformaciones (como aplicar logaritmos) antes de entrenar nuestros modelos de Regresión.

```
# 3.1.4 Normal QQ-Plot (Validación de Normalidad del Precio)
ggplot(df_eda, aes(sample = price)) +
  stat_qq(color = "steelblue", alpha = 0.5) +
  stat_qq_line(color = "red", linetype = "dashed", size = 1) +
  theme_minimal() +
  labs(title = "Normal QQ-Plot del Precio",
       subtitle = "La separación de la línea roja indica que el precio NO es normal (Asimetría positiva)",
       x = "Cuantiles Teóricos",
       y = "Cuantiles de la Muestra (Precio)")
```

Normal QQ-Plot del Precio

La separación de la línea roja indica que el precio NO es normal (Asimetría positiva)



3.2. Análisis Bivariante

3.2.1 Matriz de Correlación

Aquí detectamos la Multicolinealidad

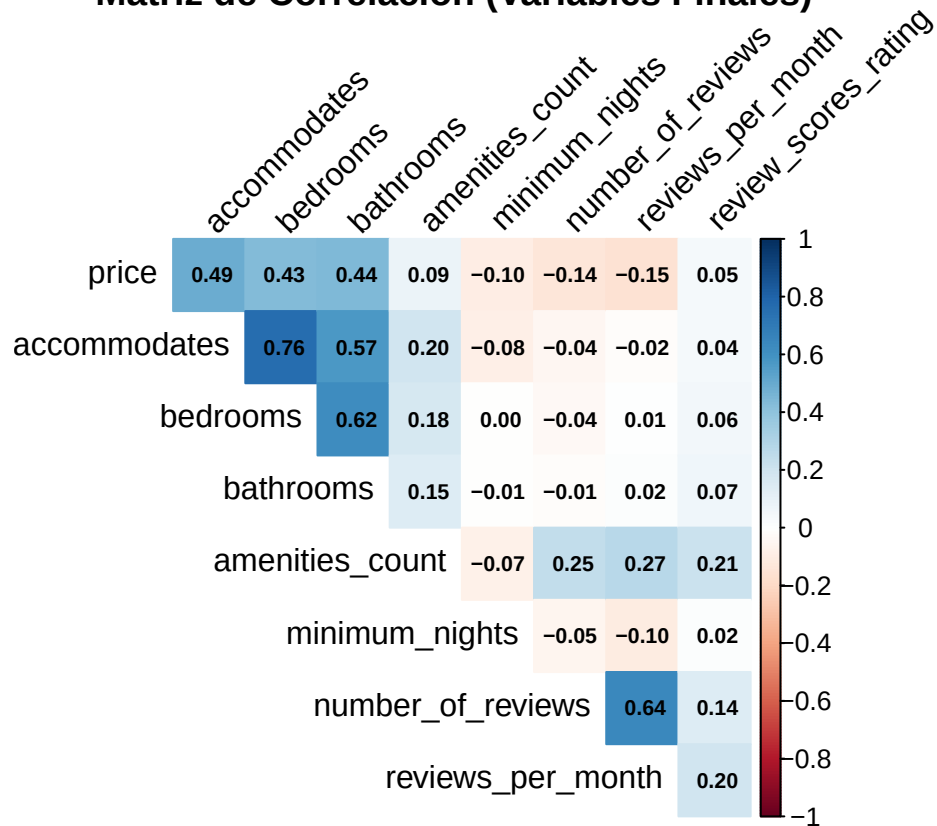
```
# 3.2.1 Matriz de Correlación (Usando las nuevas variables numéricas)
library(corrplot)

# Seleccionamos las numéricas clave del nuevo dataset
nums <- df_eda %>%
  select(price, accommodates, bedrooms, bathrooms, # Tu variable limpia
         amenities_count, minimum_nights,          # Tus variables nuevas
         number_of_reviews, reviews_per_month,
         review_scores_rating) %>%
  na.omit()

M <- cor(nums)

corrplot(M, method = "color", type = "upper",
         addCoef.col = "black", number.cex = 0.7, # Tamaño letra números
         tl.col = "black", tl.srt = 45,
         diag = FALSE,
         title = "Matriz de Correlación (Variables Finales)",
         mar = c(0,0,1,0)) # Ajuste de margen para el título
```

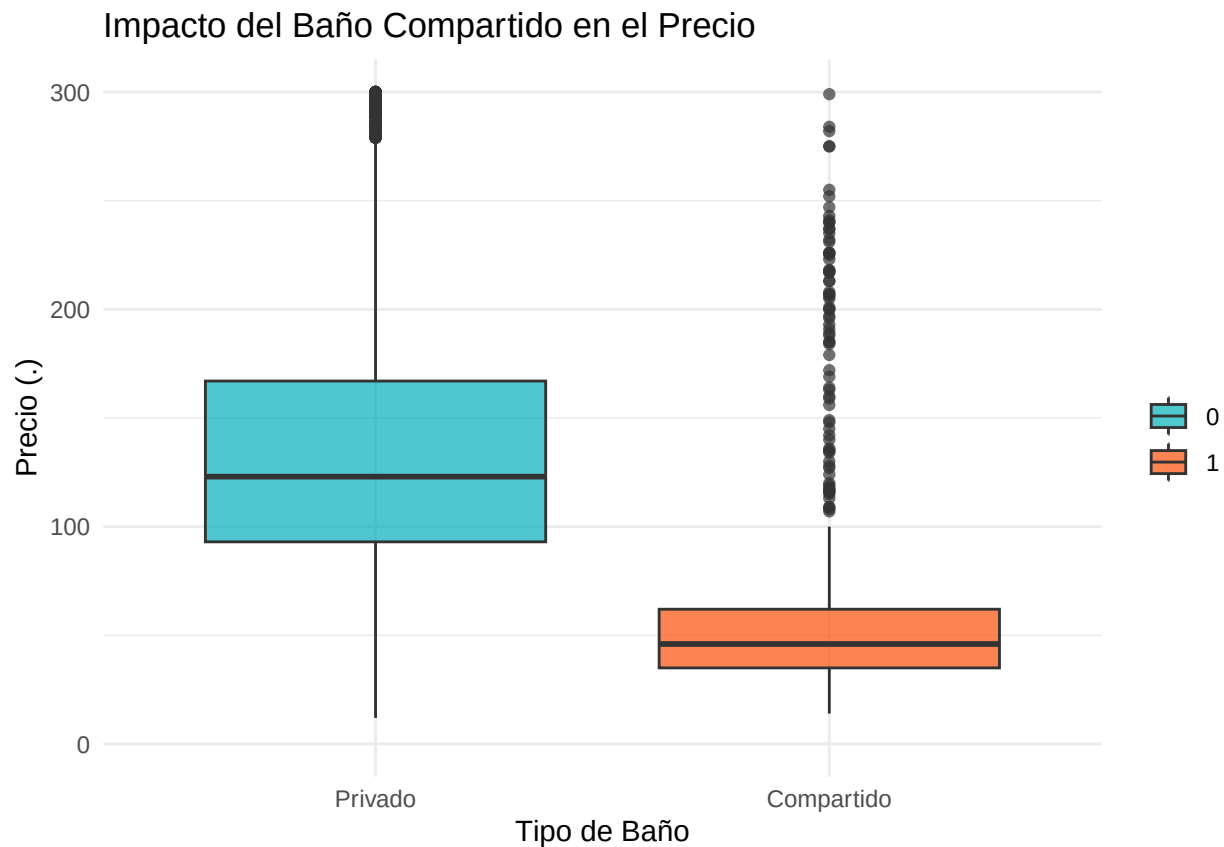
Matriz de Correlación (Variables Finales)



3.2.2 Impacto del Baño Compartido

```
# 3.2.2 Impacto del Baño Compartido
# Esto valida visualmente si 'bath_shared' afecta al precio
ggplot(df_eda, aes(x = as.factor(bath_shared), y = price, fill = as.factor(bath_shared))) +
  geom_boxplot(alpha = 0.7) +
  scale_x_discrete(labels = c("0" = "Privado", "1" = "Compartido")) +
  scale_fill_manual(values = c("#00AFBB", "#FC4E07")) +
  theme_minimal() +
  ylim(0, 300) + # Zoom para ver mejor las cajas (ignorando mansiones)
  labs(title = "Impacto del Baño Compartido en el Precio",
       x = "Tipo de Baño", y = "Precio (€)", fill = "")
```

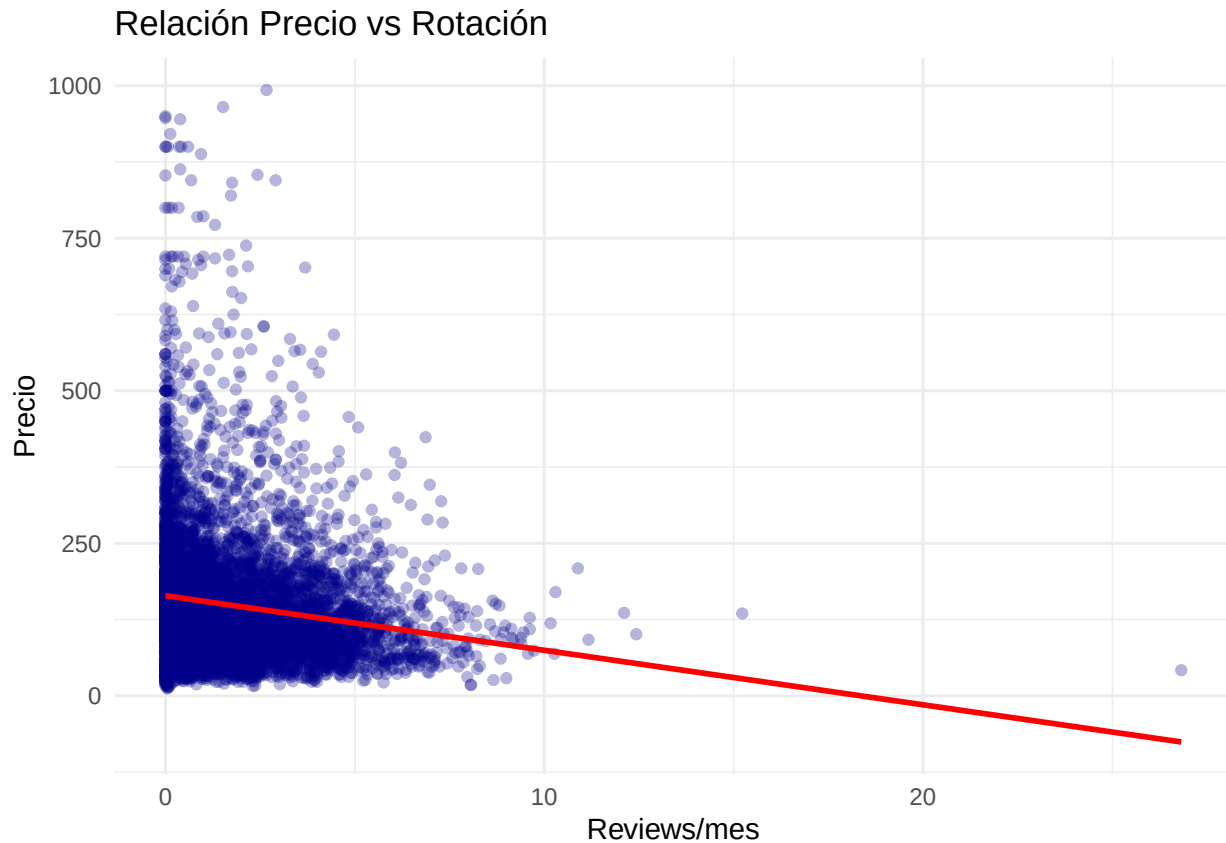
```
## Warning: Removed 469 rows containing non-finite outside the scale range (`stat_boxplot()`).
```

3.2.3 Scatterplot Price vs Reviews

```
# 3.2.3 Precio vs Reviews (Rotación)
ggplot(df_eda, aes(x = reviews_per_month, y = price)) +
  geom_point(alpha = 0.3, color = "darkblue") +
  geom_smooth(method = "lm", color = "red", se = FALSE) +
  theme_minimal() +
  labs(title = "Relación Precio vs Rotación", x = "Reviews/mes", y = "Precio")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

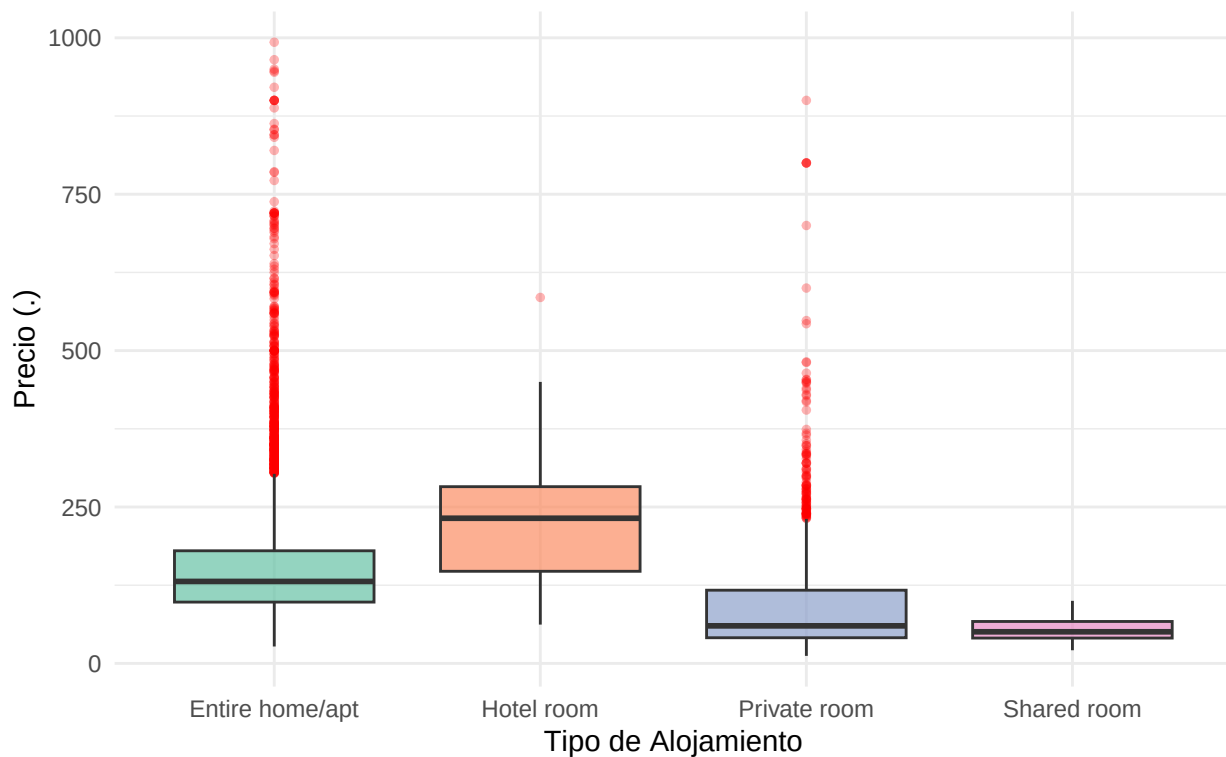


3.2.4 Gráfico Multivariante: Boxplot Paralelo (Categorica + Cuantitativa)

```
# 3.2.4 Boxplot Paralelo: Precio vs Tipo de Alojamiento
ggplot(df_eda, aes(x = room_type, y = price, fill = room_type)) +
  geom_boxplot(alpha = 0.7, outlier.color = "red", outlier.size = 1, outlier.alpha = 0.3) +
  theme_minimal() +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Boxplot Paralelo: Precios según Tipo de Alojamiento",
       subtitle = "Muestra la dispersión y los outliers (puntos rojos) de cada categoría",
       x = "Tipo de Alojamiento",
       y = "Precio (€)") +
  theme(legend.position = "none")
```

Boxplot Paralelo: Precios según Tipo de Alojamiento

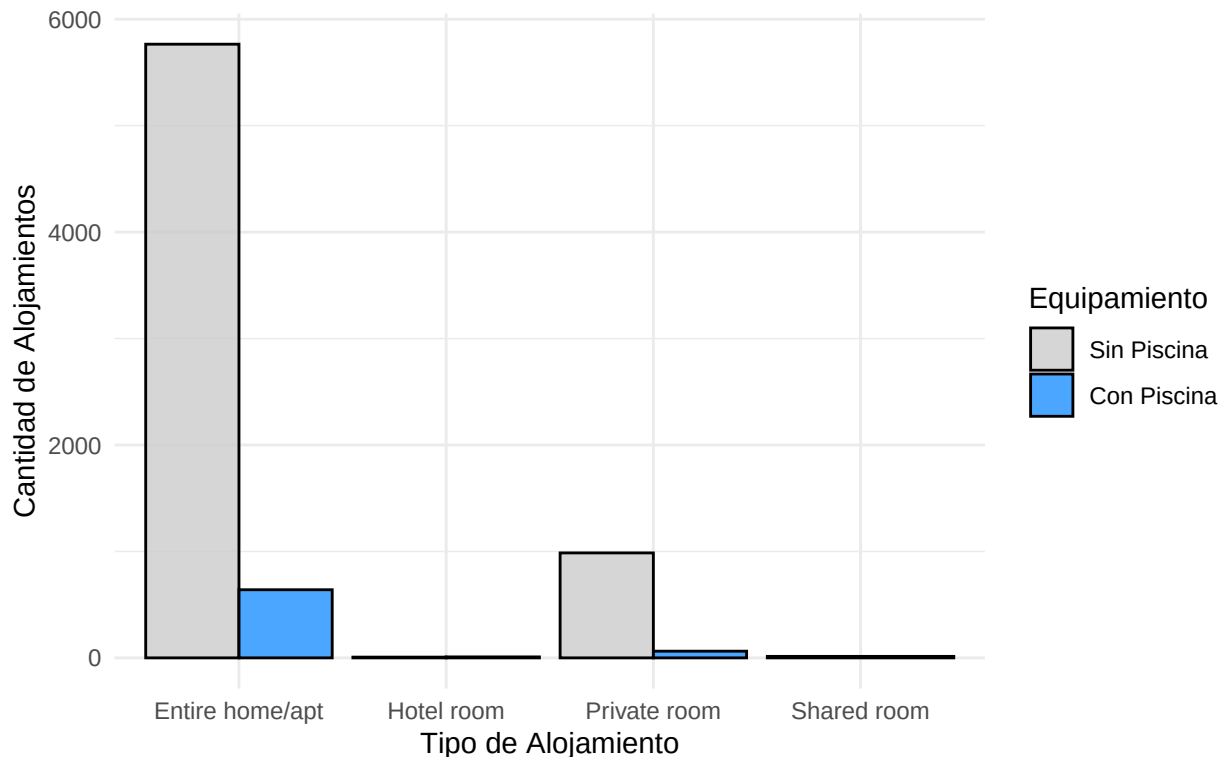
Muestra la dispersión y los outliers (puntos rojos) de cada categoría



3.2.5 Diagrama de Barras Agrupado (Dos Categóricas)

```
# Convertimos has_pool a factor solo para el gráfico
ggplot(df_eda, aes(x = room_type, fill = as.factor(has_pool))) +
  geom_bar(position = "dodge", color = "black", alpha = 0.8) + # position='dodge' las pone lado a lado
  theme_minimal() +
  scale_fill_manual(values = c("gray80", "#1E90FF"),
                    labels = c("Sin Piscina", "Con Piscina")) +
  labs(title = "Diagrama de Barras Agrupado: Distribución de Piscinas",
       subtitle = "Se observa que las piscinas son exclusivas de los Entire home/apt",
       x = "Tipo de Alojamiento",
       y = "Cantidad de Alojamientos",
       fill = "Equipamiento")
```

Se observa que las piscinas son exclusivas de los Entire home/apt



3.2.6 Pairplot: Relaciones Multivariantes

```
#install.packages("GGally")
```

```
library(GGally)
```

Seleccionamos solo las 4 variables más predictivas para no sobrecargar el gráfico

```
df_pair <- df_eda %>%
  select(price, accommodates, amenities_count, reviews_per_month)
```

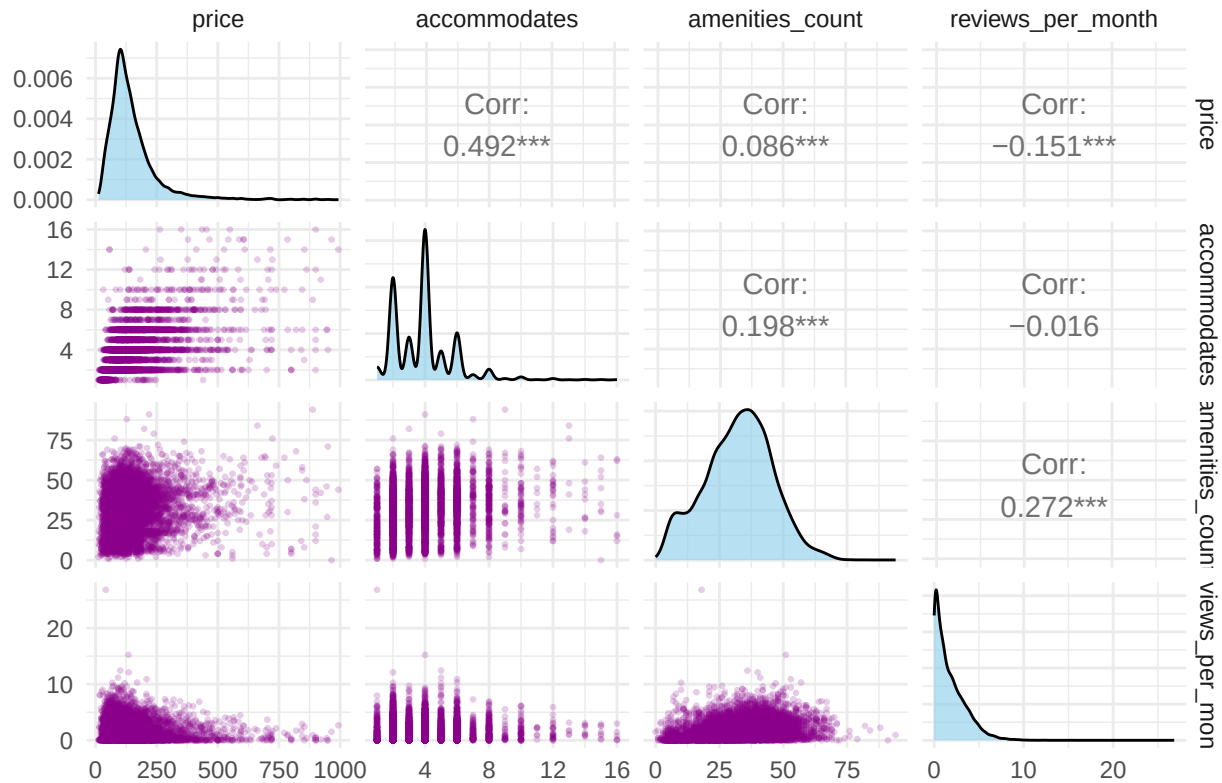
```
# Generamos el Pairplot (tarda un par de segundos en procesar)
```

```
ggpairs(df_pair,
  title = "Pairplot: Relaciones Clave del Mercado",
  lower = list(continuous = wrap("points", alpha = 0.2, size=0.5, color="darkmagenta")),
  diag = list(continuous = wrap("densityDiag", fill="skyblue", alpha=0.6))) +
  theme_minimal()
```

```
## plot: [1, 1] [====>-----]
## plot: [1, 2] [=====>-----]
## plot: [1, 3] [=====>-----]
## plot: [1, 4] [=====>-----]
## plot: [2, 1] [=====>-----]
## plot: [2, 2] [=====>-----]
## plot: [2, 3] [=====>-----]
## plot: [2, 4] [=====>-----]
## plot: [3, 1] [=====>-----]
## plot: [3, 2] [=====>-----]
```

```
## plot: [3, 3] [=====>-----]
## plot: [3, 4] [=====>-----]
## plot: [4, 1] [=====>-----]
## plot: [4, 2] [=====>-----]
## plot: [4, 3] [=====>-----]
## plot: [4, 4] [=====>-----]
```

Pairplot: Relaciones Clave del Mercado



4. Análisis Grupal

4.1. Segmentación del Mercado

Hipótesis planteada:

¿Podemos identificar clústeres de comportamiento que demuestren que el mercado de Airbnb se estructura por modelos de negocio y estrategias financieras y no por barrios?

Queremos comprobar si la realidad del mercado ignora las fronteras del mapa. Por ejemplo, buscamos identificar si existen auténticos “hoteles encubiertos” o alojamientos de lujo operando en barrios periféricos y de rentas más bajas; o si, por el contrario, encontramos habitaciones de bajo coste y altísima rotación camufladas en los distritos supuestamente más exclusivos de la ciudad.

Para descubrir estas realidades ocultas tras los precios, la disponibilidad y el volumen de reservas, aplicaremos técnicas de aprendizaje automático no supervisado (**Clustering K-Means**). Al no indicarle al algoritmo en qué barrio está cada piso, obligaremos a la matemática a agrupar los alojamientos basándose **única y exclusivamente en su rendimiento económico y operativo**.

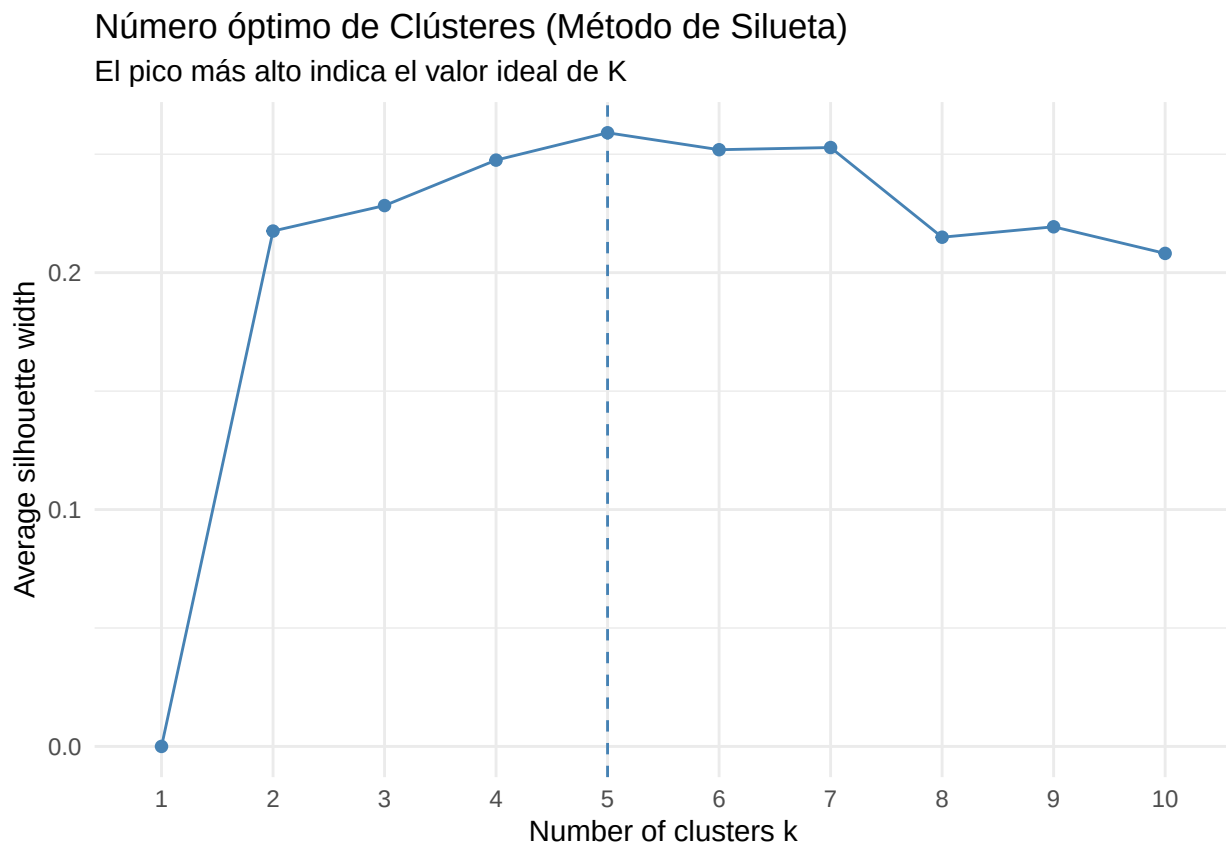
```
library(cluster)
library(factoextra)
```

1. SELECCIÓN DE VARIABLES

```
df_cluster <- df_eda %>%  
  select(id, neighbourhood_group_cleansed, neighbourhood_cleansed,  
         price, accommodates, reviews_per_month,  
         availability_365, calculated_host_listings_count,  
         latitude, longitude) %>%  
  drop_na() # K-Means da error si hay algún NA residual en estas columnas  
  
# elegimos solo las variables matemáticas para el kmeans  
datos_kmeans <- df_cluster %>%  
  select(price, accommodates, reviews_per_month,  
         availability_365, calculated_host_listings_count)  
  
# estandarizamos para que el precio-hasta 1000 no pise a reviews_per_month(0-5)  
datos_escalados <- scale(datos_kmeans)
```

2. ANÁLISIS DE SILHOUETTE

```
fviz_nbclust(datos_escalados, kmeans, method = "silhouette") +  
  theme_minimal() +  
  labs(title = "Número óptimo de Clústeres (Método de Silueta)",  
        subtitle = "El pico más alto indica el valor ideal de K")
```



3. APLICAMOS EL ALGORITMO K-MEANS k=5

```
set.seed(123)  
modelo_km_5 <- kmeans(datos_escalados, centers = 5, nstart = 25)  
  
# Añadimos la columna al dataset
```

```
df_cluster$Cluster <- as.factor(modelo_km_5$cluster)
```

```
perfiles_cluster_5 <- df_cluster %>%
  group_by(Cluster) %>%
  summarise(
    Precio_Medio = round(mean(price), 1),
    Plazas_Medias = round(mean(accommodates), 1),
    Rotacion_Mes = round(mean(reviews_per_month), 2),
    Disponibilidad = round(mean(availability_365), 0),
    Anuncios_por_Host = round(mean(calculated_host_listings_count), 1),
    Total_Pisos = n()
  ) %>%
  arrange(Precio_Medio)

print(perfiles_cluster_5)
```

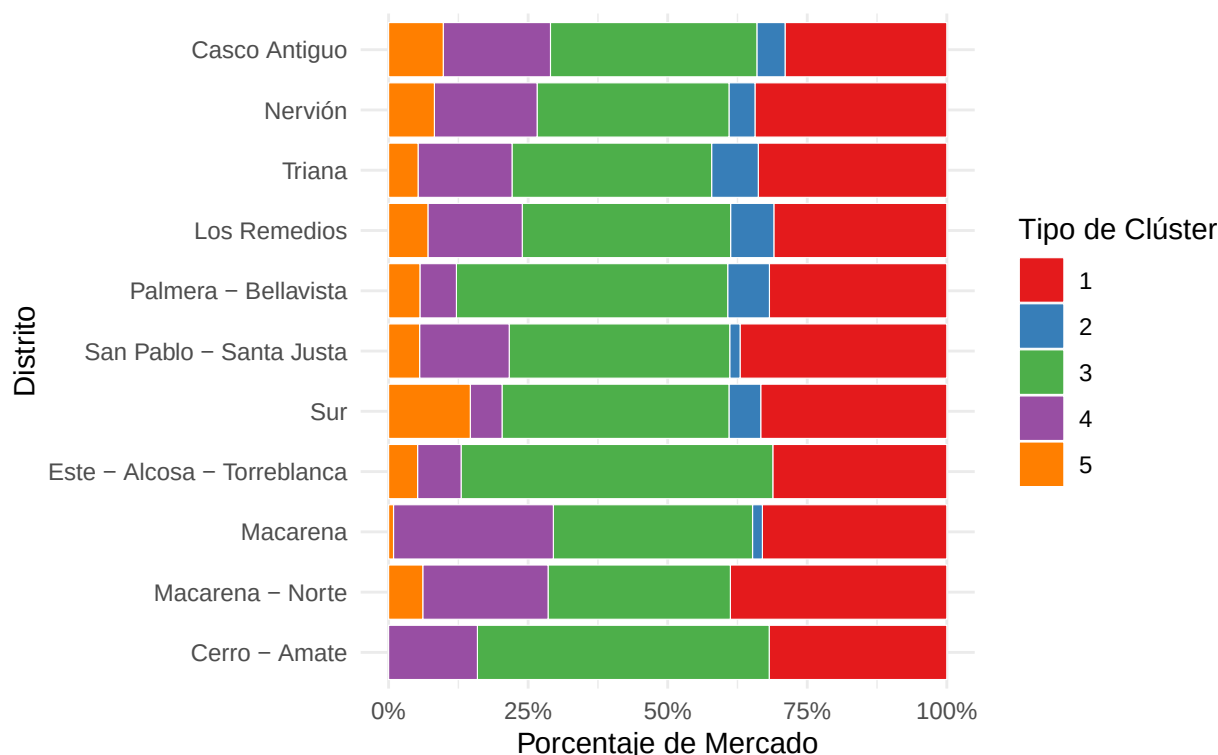
```
## # A tibble: 5 x 7
##   Cluster Precio_Medio Plazas_Medias Rotacion_Mes Disponibilidad Anuncios_por_Host Total_Pisos
##   <fct>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <int>
## 1 4          110.        3.5        4.49        160         8.9       1385
## 2 1          120.        3.5        1.05         89         9.7       2279
## 3 3          138.        3.6        1.01        284        16.9       2789
## 4 2          149.         4         1.83        217        160.        390
## 5 5          380.        7.1        1.38        214        18.7       641
```

5. GRÁFICO BARRIO VS CLÚSTER

```
ggplot(df_cluster, aes(x = reorder(neighbourhood_group_cleansed, price, median), fill = Cluster)) +
  geom_bar(position = "fill", color = "white", linewidth = 0.2) +
  coord_flip() + # Barras horizontales para leer bien los nombres
  scale_y_continuous(labels = scales::percent) +
  scale_fill_brewer(palette = "Set1") + # Cambia los colores si quieres
  theme_minimal() +
  labs(title = "Estudio de Mercado: ¿Qué se esconde en cada Barrio?",
       subtitle = "Proporción de tipos de negocio (Clústeres) por distrito",
       x = "Distrito",
       y = "Porcentaje de Mercado",
       fill = "Tipo de Clúster")
```

Estudio de Mercado: ¿Qué se esconde en cada Barrio?

Proporción de tipos de negocio (Clústeres) por distrito



Para que nuestra segmentación de mercado sea objetiva, hemos dejado que la propia estructura de los datos nos revele su división natural, en lugar de forzar al algoritmo a buscar un número arbitrario de grupos (como podrían ser los 11 distritos administrativos que compararemos más adelante).

Para lograr esto, hemos aplicado el **Análisis de Silueta (Silhouette Index)**. Esta técnica de validación evalúa el rendimiento del clustering midiendo dos factores clave para cada posible valor de k :

1. **Cohesión:** Qué tan similares son los alojamientos dentro de un mismo grupo.
2. **Separación:** Qué tan bien diferenciados están de los alojamientos de los demás grupos.

El algoritmo nos indicó que el pico máximo de rendimiento (el punto donde los grupos son más sólidos y diferentes entre sí) se encuentra exactamente en $k = 5$.

Aplicación al Modelo K-Means:

Basándonos en este criterio estrictamente matemático, hemos entrenado nuestro modelo definitivo de K-Means forzando la creación de 5 centros. Esto nos garantiza que los cinco perfiles resultantes no son una agrupación casual, sino la **estructura económica real y estadísticamente robusta** que subyace en el mercado inmobiliario turístico de la ciudad.

A continuación, procedemos a analizar la naturaleza de estos 5 clústeres descubiertos.

- **Morado y Verde** (podríamos denominarlos como Lujo y Macro-Agencias): **disposición céntrica.**
 - Representan hasta el 20% del mercado en distritos ricos y céntricos (Casco Antiguo, Nervión).
 - Desaparecen casi por completo en la periferia (Cerro-Amate, Macarena).

Posible motivo: El capital profesional y premium solo invierten donde está garantizado el turista internacional de alto presupuesto.

- **Rojo** (podríamos denominarlos como Alquiler Turístico Estrella): **el modelo transversal.**

- Es la base de Airbnb: domina casi el 50% de la oferta en *todos* los distritos por igual.

Posible motivo: se trata de pequeños propietarios que buscan alta rentabilidad mediante rotación rápida, en toda Sevilla, sin importar la lejanía al centro.

- **Azul** (podríamos denominarlos como Media y Larga Estancia): **refugio de la periferia.**

- Su proporción crece notablemente en barrios alejados del circuito turístico (como Macarena-Norte o Cerro-Amate).

Posible motivo: Al no recibir turistas de fin de semana, los dueños adaptan su negocio alquilando a trabajadores temporales o estudiantes (pisos disponibles todo el año, pero con estancias más largas y menos reseñas).

Una vez consolidados nuestros 5 modelos de negocio mediante el índice de silueta, nos surgió la curiosidad de segmentar haciendo obligatoria la k igual al número de distritos definidos, para responder definitivamente a nuestra hipótesis principal: *¿Es verdaderamente el modelo de negocio independiente de la zona, o estamos ignorando la importancia de los barrios?*

Para analizar esta cuestión, diseñamos un experimento de control. El planteamiento fue el siguiente: si el comportamiento económico de un piso dependiera puramente de su ubicación, forzando al algoritmo a generar exactamente **11 clústeres** (coincidiendo con los 11 distritos administrativos de Sevilla), debería ser capaz de “montar” el mapa político de la ciudad basándose solo en los datos.

El **objetivo** de esta comparativa es comprobar si al darle al algoritmo el mismo número de opciones que distritos, cada barrio se asignaba a un clúster único o si por el contrario, los distritos se fragmentaban.

Esto es clave porque nos permite demostrar empíricamente si las fronteras que vemos en los mapas (los códigos postales) tienen algún valor real a nivel financiero y operativo dentro de la plataforma Airbnb, o si son meramente un constructo administrativo que el mercado turístico ignora por completo.

A continuación, analizamos los resultados visuales de este experimento:

```
set.seed(123)
modelo_km_11 <- kmeans(datos_escalados, centers = 11, nstart = 25)

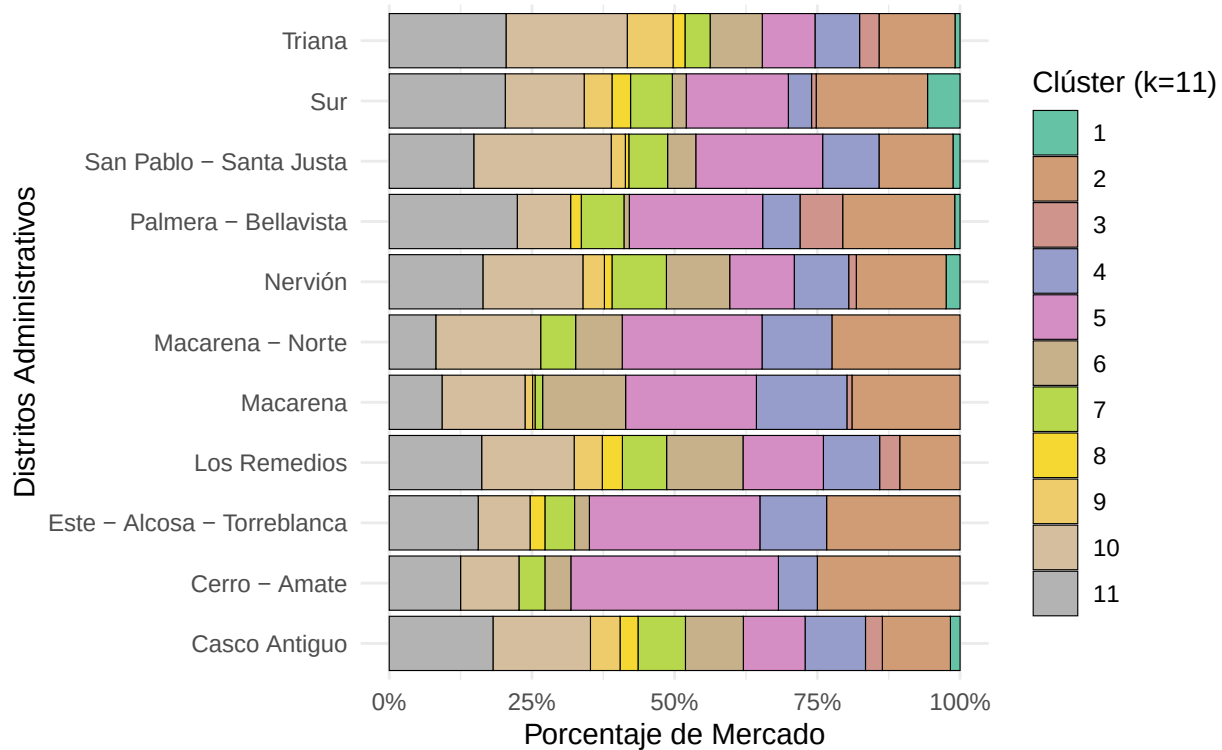
## Warning: did not converge in 10 iterations

df_cluster$Cluster_11 <- as.factor(modelo_km_11$cluster)

# GRÁFICO: DISTRITOS VS CLÚSTERES
ggplot(df_cluster, aes(x = neighbourhood_group_cleansed, fill = Cluster_11)) +
  geom_bar(position = "fill", color = "black", linewidth = 0.2) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  # Usamos una paleta con muchos colores porque ahora tenemos 11 grupos
  scale_fill_manual(values = colorRampPalette(RColorBrewer::brewer.pal(8, "Set2"))(11)) +
  theme_minimal() +
  labs(title = "Experimento k=11:",
       subtitle = "¿Coinciden los clústeres con los barrios administrativos?",
       x = "Distritos Administrativos",
       y = "Porcentaje de Mercado",
       fill = "Clúster (k=11)") +
  theme(legend.position = "right")
```

Experimento k=11:

¿Coinciden los clústeres con los barrios administrativos?



El Mito de los barrios (k=11)

Para validar la segmentación de 5 modelos de negocio, se planteó una prueba de refutación:

*Si forzamos al algoritmo a buscar 11 clústeres,
¿será capaz de mapear exactamente los 11 distritos de Sevilla?*

Los resultados del gráfico k=11 rechaza esta hipótesis claramente:

- **Hiper-fragmentación por distrito:** En lugar de obtener barras de un solo color sólido (lo que indicaría que cada barrio tiene un comportamiento único y uniforme), observamos que todos los distritos están fragmentados en diferentes clusters matemáticos.

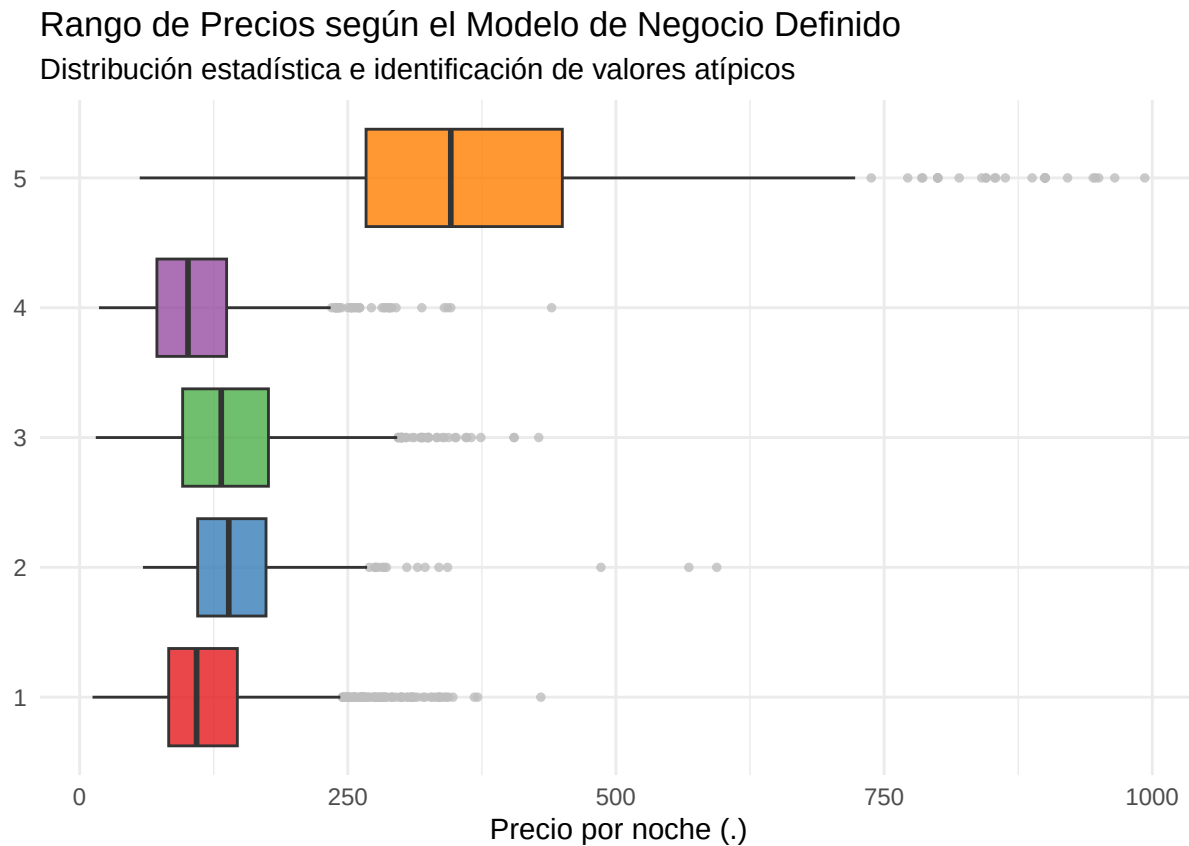
Conclusión: La gran mezcla de clusters en el experimento de $k = 11$ demuestra que **la geografía administrativa no dicta el modelo de negocio**. Un alojamiento no se comporta de una manera determinada simplemente por pertenecer al distrito de “San Pablo” o “Los Remedios”. Las verdaderas fronteras del mercado en Sevilla son invisibles. Podríamos deducir que están definidas por estrategias de inversión (similares a los 5 perfiles definidos en el primer experimento), las cuales conviven y compiten de forma superpuesta incluso en las mismas calles de Sevilla.

4.1.1. Visualización avanzada de clustering

Hemos analizado un poco más en detalle las relaciones entre diferentes variables que consideramos interesantes ampliar su estudio, con los 2 gráficos siguientes:

Análisis de Distribución de Precios según el modelo de Negocio

```
# GRÁFICO 2: DISTRIBUCIÓN DE PRECIOS POR CLUSTER (K=5)
ggplot(df_cluster, aes(x = Cluster, y = price, fill = Cluster)) +
  geom_boxplot(alpha = 0.8, outlier.color = "grey", outlier.size = 1) +
  scale_fill_brewer(palette = "Set1") +
  coord_flip() + # Barras horizontales
  theme_minimal() +
  labs(title = "Rango de Precios según el Modelo de Negocio Definido",
       subtitle = "Distribución estadística e identificación de valores atípicos",
       x = "",
       y = "Precio por noche (€)") +
  theme(legend.position = "none")
```



El análisis de dispersión de precios a través de los boxplots revela la estructura financiera que sostiene cada modelo de negocio en Sevilla:

- **1. La Guerra de Precios en la base del mercado (Rojo, Azul y Verde):** Los clusteres de *Alta Rotación*, *Particular Ocasional* y *Media/Larga Estancia* presentan medianas muy similares (todas rondando los 100€ - 120€) y con cajas muy compactas.
 - Esto demuestra que en el mercado estándar hay mucha competencia y los precios están estandarizados. Como no pueden competir subiendo el precio (el mercado no lo aceptaría), los anfitriones se diferencian por su estrategia de disponibilidad y volumen de reservas.
- **2. El recargo por la Profesionalización (Macro-Agencias - Morado):** A diferencia de los particulares, la caja de las *Macro-Agencias* está claramente desplazada hacia la derecha, con una mediana cercana a los 150€. Además, es una caja relativamente estrecha, lo que indica bastante control en sus tarifas.
 - Los grandes inversores no operan en la base del mercado low-cost. Su estrategia se basa en

ofrecer un estándar de calidad, diseño o servicio que les permite cobrar un sobreprecio constante, asegurando así su margen de rentabilidad a pesar de ese sobrecoste.

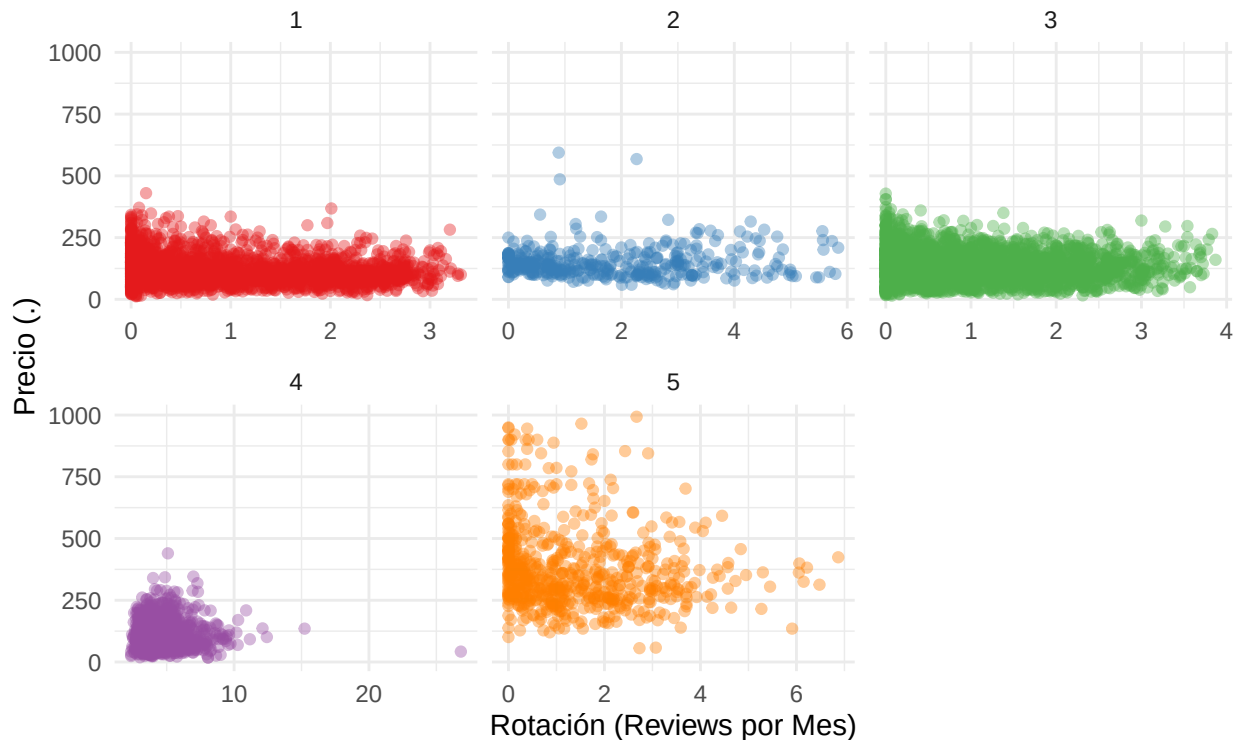
- **3. La extrema volatilidad del Lujo (Naranja):** Es la caja que más destaca visualmente. Su mediana se dispara entorno a los 350€, pero lo más sorprendente es su anchura (rango intercuartílico) y la gran cantidad de valores atípicos (los puntos grises) que se estiran hasta los 1.000€.
 - El mercado de lujo está muy poco estandarizado. Mientras que un piso normal tiene un precio tope definido, en el lujo “no hay límite”. El precio ya no depende del mercado base, sino de características exclusivas únicas (vistas directas a la Giralda, piscinas privadas, capacidad para 10 personas, etc.).

Análisis de Estrategia Comercial: Matriz Precio vs. Rotación:

```
# GRÁFICO 3: MATRIZ DE RENTABILIDAD
ggplot(df_cluster, aes(x = reviews_per_month, y = price, color = Cluster)) +
  geom_point(alpha = 0.4) +
  facet_wrap(~Cluster, scales = "free_x") +
  scale_color_brewer(palette = "Set1") +
  theme_minimal() +
  labs(title = "Matriz Precio vs Rotación",
       subtitle = "Comportamiento del volumen de reservas frente al precio",
       x = "Rotación (Reviews por Mes)",
       y = "Precio (€)") +
  theme(legend.position = "none")
```

Matriz Precio vs Rotación

Comportamiento del volumen de reservas frente al precio



Al desglosar la relación entre el precio por noche, el margen, y las reseñas mensuales, volumen, a través de un gráfico de dispersión, podemos observar visualmente las distintas “fórmulas de rentabilidad” que emplean

los anfitriones de Sevilla:

- **Estrategia de Volumen (El panel Rojo - Alta Rotación):** El gráfico demuestra visualmente por qué este cluster recibe su nombre. Mientras el resto de modelos rara vez superan las 4 o 5 reseñas al mes, este grupo genera una cola extrema que llega hasta las 20 reseñas mensuales. Su estrategia está clara: mantienen precios estrictamente bajos, concentrados en el límite inferior del eje Y, para maximizar la ocupación de fin de semana.
- **Estrategia de Margen (El panel Naranja - Lujo):** Representa exactamente el modelo opuesto al anterior. La nube de puntos tiene forma vertical. Sacrifican totalmente el volumen de reservas (eje X muy corto, casi todo concentrado entre 0 y 2 reseñas/mes) para disparar el precio de la noche, con precios tan distribuidos llegando incluso a los 1.000€. Su rentabilidad se basa en pocas reservas de muchísimo valor.
- **La “Zona de Confort” del mercado (Paneles Azul, Verde y Morado):** Los *Particulares*, la *Media Estancia* y las *Macro-Agencias* tienen nubes de puntos densas, es decir, operan en el cuadrante inferior izquierdo (precios moderados y rotación baja/media). Su éxito financiero no depende de este gráfico, sino de la variable de “Disponibilidad Anual” para mantener los pisos activos el máximo tiempo posible.

Conclusiones del K-Means: Estudio del Mercado por Distritos

A través de la segmentación por clustering (K-Means) y el análisis de las variables operativas y financieras, hemos dado respuesta a nuestra hipótesis inicial, contrastando que la realidad del mercado de alojamientos en Sevilla es mucho más compleja que el simple “estándar del barrio”.

Las conclusiones clave de nuestro estudio son:

1. El Mito de la Geografía Administrativa (K=5 vs K=11) El experimento de refutación forzando 11 clústeres demostró una hiper-fragmentación en todos los distritos, confirmando que las fronteras administrativas (los barrios) no dictan el comportamiento económico de un inmueble. En su lugar, hemos identificado que el mercado real se divide en **5 modelos de negocio** (*Alta Rotación*, *Particular Ocasional*, *Media/Larga Estancia*, *Macro-Agencias* y *Lujo*). Estos perfiles compiten y se superponen en las mismas calles: un piso en el centro puede ser una “habitación para salir del paso” y el de al lado un ático gestionado por un fondo de inversión con lógicas financieras completamente opuestas.

2. Polarización de las Estrategias de Rentabilidad (Volumen vs. Margen) El análisis de la matriz de rotación frente al precio revela que los anfitriones adoptan estrategias comerciales polarizadas para maximizar su rentabilidad:

- **Estrategia de Volumen:** es dominada por el cluster de *Alta Rotación*. Mantienen precios estrictamente bajos para forzar una ocupación masiva y constante (hasta 20 reseñas mensuales), rentabilizando el inmueble a base de cantidad de huéspedes.
- **Estrategia de Margen:** es liderada por el cluster de *Lujo*. Renuncian por completo al volumen alto, para disparar el precio por noche medio, asegurando la rentabilidad y viabilidad del negocio con muy pocas reservas de altísimo valor.

3. Las Tres “Ligas” de precios del mercado La distribución estadística con boxplots nos confirma que no existe una guerra de precios única, sino tres estratos bien diferenciados:

- Una **base hipercompetitiva** (Particulares y Alta Rotación), donde los precios están fuertemente estandarizados y los márgenes de subida son mínimos.
- Una **capa profesional** (Macro-Agencias), que logra imponer un sobreprecio constante (premium) gracias a su estandarización de servicios extra.
- Un **techo impensable** (Lujo), caracterizado por una altísima volatilidad y dispersión, donde el precio no depende del mercado base sino de la exclusividad del apartamento.

Resolución de la Hipótesis: Definitivamente, **sí podemos identificar clusteres que rompen con el estándar de su zona.** Nuestro análisis demuestra que detrás de un simple “precio por noche” en Airbnb se esconde una compleja estructura. El mercado no se define por “barrios caros y baratos”, sino por estrategias de inversión, demostrando que la profesionalización, la disponibilidad y la rotación son los verdaderos motores del éxito inmobiliario turístico en la ciudad.

4.2. Determinantes del Precio

¿Somos capaces de calcular el precio justo de cada alojamiento y detectar matemáticamente qué pisos están inflando su valor aprovechando el turismo?

```
library(tidyverse)
library(broom)
library(car)
library(lmtest)

listings_4_2 <- read.csv("../csv/listings_preprocesado.csv")
```

4.2.1 Entrenamiento del modelo de regresión

Para dar respuesta a esta pregunta hemos decidido utilizar un modelo de **Regresión Lineal Generalizado (GLM)** con distribución Gamma y enlace logarítmico. Este enfoque nos permite evitar que el modelo prediga precios negativos y para entender que los extras como son la piscina o ubicación no suman una cantidad fija sino un porcentaje sobre el precio base.

```
# Modelo explicativo base
modelo_precio <- glm(price ~ neighbourhood_cleansed +
  room_type +
  accommodates +
  bathrooms +
  has_pool +
  has_ac +
  review_scores_rating,
  family = Gamma(link = "log"),
  data = listings_4_2)

summary(modelo_precio)
```

```
##
## Call:
## glm(formula = price ~ neighbourhood_cleansed + room_type + accommodates +
##     bathrooms + has_pool + has_ac + review_scores_rating, family = Gamma(link = "log"),
##     data = listings_4_2)
##
## Coefficients:
##
## (Intercept)
## neighbourhood_cleansedAlfalfa
## neighbourhood_cleansedAmate
## neighbourhood_cleansedan Roque
## neighbourhood_cleansedArenal
## neighbourhood_cleansedAvda. de la Paz
## neighbourhood_cleansedBami
## neighbourhood_cleansedBarrio León
```

	Estimate	Std. Err
	4.033642	0.413
	0.640557	0.402
	-0.590712	0.449
	0.446756	0.405
	0.751418	0.402
	-0.624396	0.696
	0.280685	0.421
	0.342885	0.416

## neighbourhood_cleansedBda. de Pineda	0.534745	0.568
## neighbourhood_cleansedBda. Pino Montano	0.113685	0.492
## neighbourhood_cleansedBegoña, Santa Catalina	-0.257628	0.434
## neighbourhood_cleansedBellavista	0.226510	0.417
## neighbourhood_cleansedCampos de Soria	-0.307573	0.518
## neighbourhood_cleansedCarretera de Carmona, María Auxiliadora, Fontanal	0.253370	0.412
## neighbourhood_cleansedCisneo Alto, Santa María de Graccia	-0.073903	0.518
## neighbourhood_cleansedCiudad Jardín	0.142680	0.431
## neighbourhood_cleansedColores, Entreparkes	0.177189	0.415
## neighbourhood_cleansedConsolación	-0.186322	0.475
## neighbourhood_cleansedCruz Roja, Capuchinos	0.075899	0.409
## neighbourhood_cleansedDoctor Barraquer, G. Renfe, Policlínico	0.588344	0.413
## neighbourhood_cleansedEl Cano, Los Bermejales	0.050103	0.463
## neighbourhood_cleansedEl Carmen	-0.108417	0.491
## neighbourhood_cleansedEl Cerro	-0.159750	0.416
## neighbourhood_cleansedEl Gordillo	-0.824505	0.696
## neighbourhood_cleansedEl Juncal, Híspalis	0.526344	0.455
## neighbourhood_cleansedEl Plantinar	0.660484	0.444
## neighbourhood_cleansedEl Porvenir	0.292720	0.411
## neighbourhood_cleansedEl Rocío	-0.228640	0.518
## neighbourhood_cleansedEl Tardón, El Carmen	0.304390	0.413
## neighbourhood_cleansedEl Torrejón, El Cerezo	-0.248151	0.424
## neighbourhood_cleansedEncarnación, Regina	0.585733	0.402
## neighbourhood_cleansedFelipe II, Los Diez Mandamientos	-0.230007	0.518
## neighbourhood_cleansedFeria	0.501823	0.402
## neighbourhood_cleansedGiralda Sur	0.196450	0.455
## neighbourhood_cleansedHeliópolis	0.205930	0.440
## neighbourhood_cleansedHermandades, La Carrasca	0.015085	0.475
## neighbourhood_cleansedHuerta de la Salud	0.302081	0.418
## neighbourhood_cleansedHuerta de Santa Teresa	0.508422	0.416
## neighbourhood_cleansedHuerta del Pilar	0.337547	0.423
## neighbourhood_cleansedJuan XXIII	-0.002652	0.429
## neighbourhood_cleansedLa Bachillera	0.122292	0.440
## neighbourhood_cleansedLa Barzola	-0.140479	0.475
## neighbourhood_cleansedLa Buhaira	0.517933	0.408
## neighbourhood_cleansedLa Calzada	0.046705	0.415
## neighbourhood_cleansedLa Corza	-0.493743	0.696
## neighbourhood_cleansedLa Florida	0.801052	0.407
## neighbourhood_cleansedLa Oliva	-0.271927	0.696
## neighbourhood_cleansedLa Palmilla, Doctor Marañón	0.076291	0.464
## neighbourhood_cleansedLa Paz, Las Golondrinas	-0.254141	0.464
## neighbourhood_cleansedLa Plata	-0.331571	0.440
## neighbourhood_cleansedLas Avenidas	-0.017791	0.492
## neighbourhood_cleansedLas Huertas	0.232846	0.492
## neighbourhood_cleansedLas Letanías	-1.089476	0.568
## neighbourhood_cleansedLeón XIII, Los Naranjos	0.109310	0.414
## neighbourhood_cleansedLos arcos	-0.384999	0.568
## neighbourhood_cleansedLos Carteros	0.097470	0.568
## neighbourhood_cleansedLos Pajaros	-0.248669	0.463
## neighbourhood_cleansedLos Príncipes, La Fontanilla	0.455634	0.518
## neighbourhood_cleansedLos Remedios	0.388677	0.404
## neighbourhood_cleansedMacarena 3 Huertas, Macarena 5	-0.614182	0.695
## neighbourhood_cleansedMuseo	0.598631	0.403
## neighbourhood_cleansedNervión	0.152383	0.407

```

## neighbourhood_cleansedno asignado 0.032831 0.415
## neighbourhood_cleansedPalacio Congresos, Urbadiez, Entrepuentes, Jardines del Eden -0.147316 0.426
## neighbourhood_cleansedPalmete 0.084847 0.455
## neighbourhood_cleansedParque Alcosa 0.335556 0.518
## neighbourhood_cleansedPedro Salvador, Las Palmeritas -0.225671 0.440
## neighbourhood_cleansedPino Flores -0.154223 0.492
## neighbourhood_cleansedPio XII -0.138543 0.431
## neighbourhood_cleansedPolígono Norte -0.517318 0.455
## neighbourhood_cleansedPolígono Sur 0.251480 0.463
## neighbourhood_cleansedPrado, Parque María Luisa 0.661780 0.491
## neighbourhood_cleansedRetiro Obrero -0.070578 0.568
## neighbourhood_cleansedRochelambert -0.336004 0.440
## neighbourhood_cleansedSan Bartolomé 0.716734 0.402
## neighbourhood_cleansedSan Bernardo 0.533163 0.405
## neighbourhood_cleansedSan Carlos, Tartessos 0.155668 0.434
## neighbourhood_cleansedSan Diego -0.011711 0.695
## neighbourhood_cleansedSan Gil 0.337060 0.403
## neighbourhood_cleansedSan Jerónimo -0.163299 0.426
## neighbourhood_cleansedSan José Obrero 0.197824 0.409
## neighbourhood_cleansedSan Julián 0.360714 0.403
## neighbourhood_cleansedSan Lorenzo 0.426642 0.403
## neighbourhood_cleansedSan Matías -0.437168 0.568
## neighbourhood_cleansedSan Pablo A y B -0.155549 0.449
## neighbourhood_cleansedSan Pablo C -0.352389 0.696
## neighbourhood_cleansedSan Pablo D y E 0.469349 0.434
## neighbourhood_cleansedSan Vicente 0.475761 0.402
## neighbourhood_cleansedSanta Aurelia, Cantábrico, Atlántico, La Romería 0.436080 0.456
## neighbourhood_cleansedSanta Catalina 0.450250 0.403
## neighbourhood_cleansedSanta Clara -0.032821 0.431
## neighbourhood_cleansedSanta Cruz 0.832407 0.402
## neighbourhood_cleansedSanta María de Ordas, San Nicolas 0.127513 0.492
## neighbourhood_cleansedSector Sur, La Palmera, Reina Mercedes 0.221329 0.413
## neighbourhood_cleansedTablada 0.592772 0.464
## neighbourhood_cleansedTabladilla, La Estrella 0.488544 0.518
## neighbourhood_cleansedTiro de Línea, Santa Genoveva 0.031506 0.427
## neighbourhood_cleansedTriana Casco Antiguo 0.474450 0.402
## neighbourhood_cleansedTriana Este 0.419936 0.403
## neighbourhood_cleansedTriana Oeste 0.425889 0.403
## neighbourhood_cleansedValdezorras -0.015023 0.492
## neighbourhood_cleansedVillegas -0.199034 0.475
## room_typeHotel room 0.394241 0.142
## room_typePrivate room -0.133434 0.022
## room_typeShared room -0.738280 0.153
## accommodates 0.109411 0.004
## bathrooms 0.102013 0.012
## has_pool 0.338659 0.023
## has_ac -0.047277 0.052
## review_scores_rating -0.032834 0.018
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Gamma family taken to be 0.3226572)
##
## Null deviance: 2636.9 on 7483 degrees of freedom

```



```
## Residual deviance: 1479.0 on 7374 degrees of freedom
## AIC: 81792
##
## Number of Fisher Scoring iterations: 7
```

El modelo ha sido entrenado con 7374 alojamientos y hemos logrado alcanzar una significancia global muy alta con un p-value de $2.2e-16$, validando de esta forma la capacidad del modelo para explicar la estructura de precios del mercado en Sevilla.

Analizando los resultados que arroja el summary nos podemos dar cuenta de un hallazgo sorprendente, cuanto menos, la mayoría de los barrios que se encuentran en la periferia como son Amate, Bellavista o Pino Montano, muestran p-values muy altos, lo que significa que no tienen un gran impacto en el precio respecto al nivel base. Sin embargo, podemos ver cómo el “Efecto Centro” es drástico. Al convertir los coeficientes logarítmicos en porcentajes ($\exp(\cdot) - 1$), descubrimos que en el barrio de Santa Cruz tenemos un sobreprecio del 129% solo por la ubicación, seguidos del Arenal con un 112%. También nos ha parecido curioso que barrios como Triana no tienen un impacto tan extremo, teniendo entre un 52% y un 60% de sobreprecio; esto nos indica que su popularidad supera a su cotización real si lo comparamos con el casco histórico

Con una devianza explicada cercana al 44%, el modelo explica casi la mitad de la varianza de precios solo con variables estructurales. El 56% restante sugiere que factores intangibles como la decoración, la calidad de las fotos o el trato del anfitrión juegan un papel decisivo en la fijación del precio final en plataformas como Airbnb.

4.2.2 Diagnóstico y validación

```
# A. Multicolinealidad (VIF)
# Verificamos que las variables no sean redundantes entre sí
vif_valores <- vif(modelo_precio)
print("--- Valores VIF (Deben ser < 5) ---")

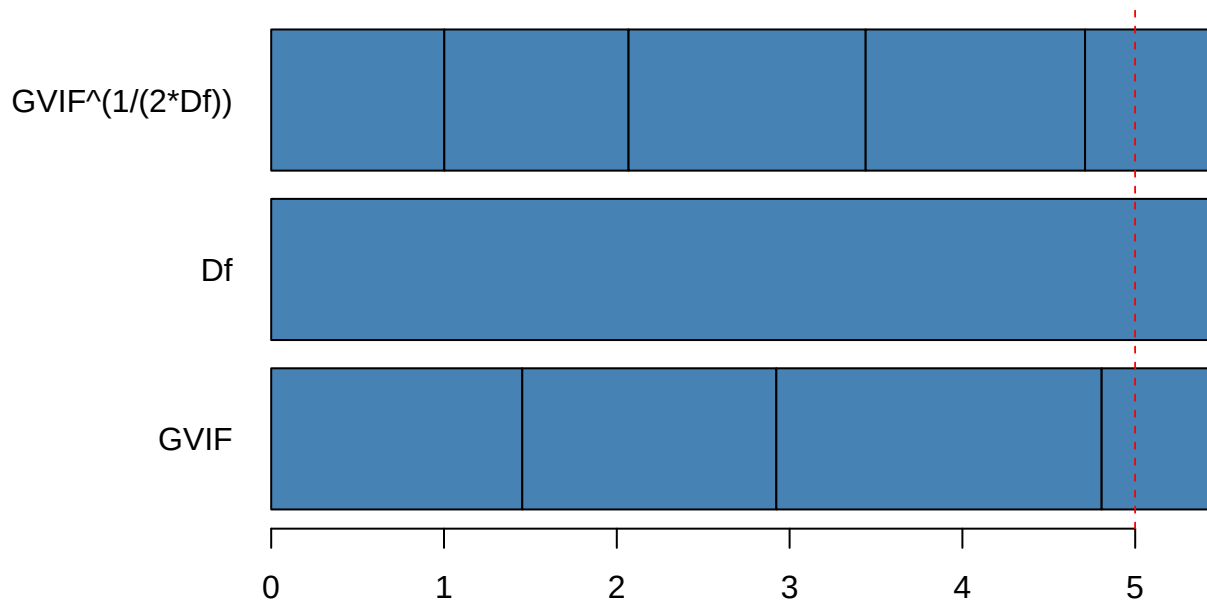
## [1] "--- Valores VIF (Deben ser < 5) ---"

print(vif_valores)

##               GVIF   Df GVIF^(1/(2*Df))
## neighbourhood_cleansed 1.452988 101      1.001851
## room_type              1.470277   3      1.066350
## accommodates           1.882531   1      1.372053
## bathrooms              1.611825   1      1.269577
## has_pool               1.078803   1      1.038654
## has_ac                 1.108862   1      1.053025
## review_scores_rating   1.041892   1      1.020731

par(mar = c(5, 8, 4, 2)) # Ajustar márgenes
barplot(vif_valores, main = "Validación: Factor de Inflación de la Varianza (VIF)",
        horiz = TRUE, col = "steelblue", xlim = c(0, 5), las = 1)
abline(v = 5, col = "red", lty = 2)
```

Validación: Factor de Inflación de la Varianza (VIF)



```
# B. Heterocedasticidad (Breusch-Pagan)
# Comprobamos si el error es constante
print("--- Test de Breusch-Pagan ---")
```

```
## [1] "--- Test de Breusch-Pagan ---"
```

```
bptest(modelo_precio)
```

```
##
## studentized Breusch-Pagan test
##
## data: modelo_precio
## BP = 226.35, df = 109, p-value = 3.166e-10
```

Antes de interpretar los coeficientes del modelo lo hemos sometido a unos diagnosticos para asegurarnos que cumple con la teoria de la regresion lineal. Un modelo con coeficientes sesgados puede llevarnos a conclusiones de negocios erroneas.

En primer lugar hemos calculado el Factor de Inflacion de la Varianza para detectar redundancia entre las variables predictoras. Como podemos ver todas las variables tienen un VIF muy bajo, que oscila entre 1,04 y 1,88. Esto nos permite llegar a la conclusion de que la independencia es robusta y que nuestras variables aportan informacion unica y no solapada al precio final.

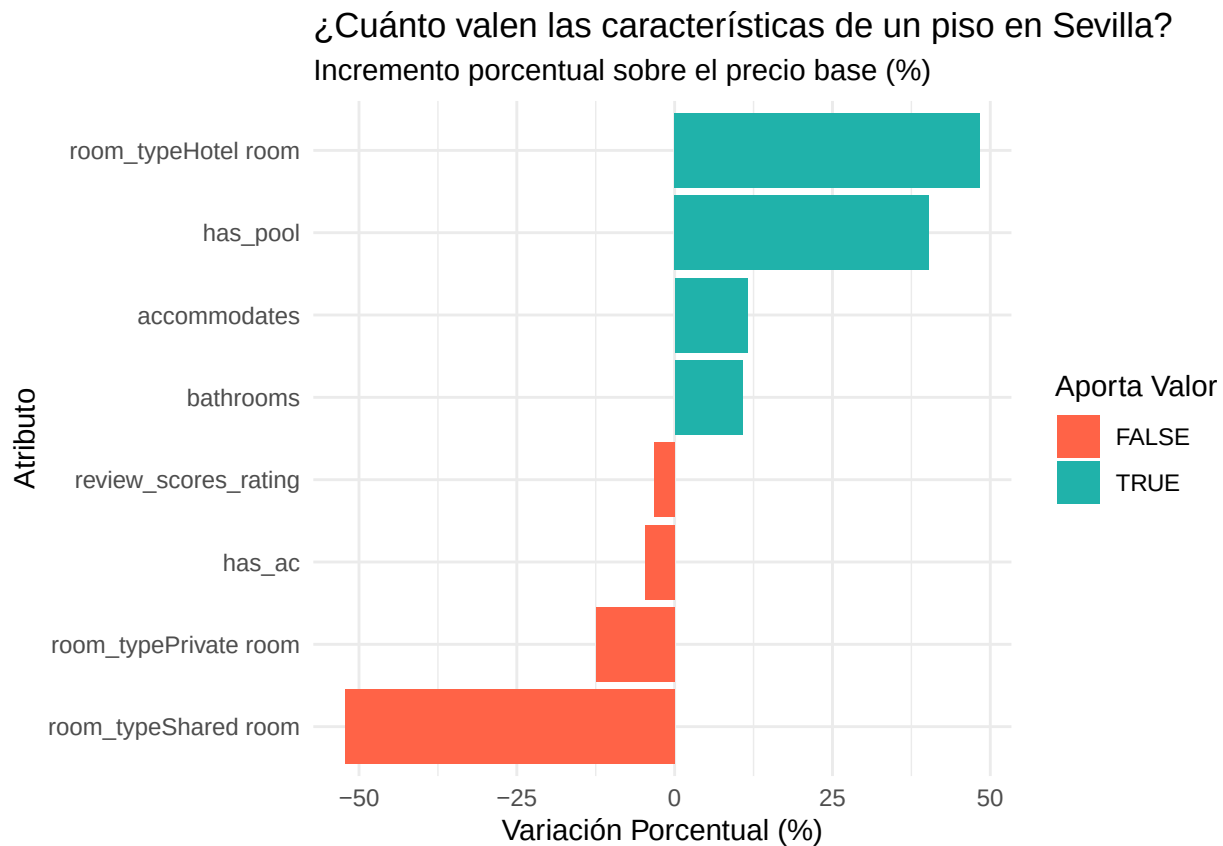
Despues hemos aplicado el test de Breusch-Pagan para confirmar si la varianza del error es constante. Dado que nos encontramos un p-value inferior a 0.05 podemos indicar que el modelo comete errores mas grandes en los pisos caros que en los baratos. No obstante, esta es la razón por la que hemos utilizado un modelo GLM con familia Gamma; este algoritmo asume y corrige matemáticamente esta falta de constancia en la varianza, garantizando que nuestros resultados y conclusiones finales sean totalmente válidos y rigurosos.

4.2.3 Interpretación del modelo

```
coeficientes <- tidy(modelo_precio) %>%
  filter(term != "(Intercept)") %>%
  filter(!str_detect(term, "neighbourhood")) %>%
```

```
mutate(impacto_pct = (exp(estimate) - 1) * 100) %>%
  arrange(desc(impacto_pct))

ggplot(coeficientes, aes(x = reorder(term, impacto_pct), y = impacto_pct, fill = impacto_pct > 0)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  theme_minimal() +
  scale_fill_manual(values = c("tomato", "lightseagreen")) + # Colores más académicos
  labs(title = "¿Cuánto valen las características de un piso en Sevilla?",
       subtitle = "Incremento porcentual sobre el precio base (%)",
       x = "Atributo", y = "Variación Porcentual (%)", fill = "Aporta Valor")
```



Tras analizar los coeficientes, podemos ver que, aunque creamos que la ubicación de un piso es el factor más determinante, no es este el único.

Como podemos ver en la gráfica anterior, el factor que más dispara el precio es la tipología de alojamiento: si es una **habitación de hotel** se añade un **sobrecoste del 48%** frente al apartamento estándar. Esto confirma la existencia de una oferta que se encuentra en manos de profesionales, las cuales operan con un esquema de precios muy superior.

También podemos observar que la piscina destaca entre los extras, aumentando el precio en un 40%. Esto nos indica que en Sevilla, la piscina no es un simple complemento, sino un activo premium indiscutible que encarece drásticamente el precio de ese alojamiento.

Por cada persona extra que cabe en el piso, el precio sube un 11,5%, y por cada baño adicional suma un 10,7% sobre el valor base. En el lado opuesto, alquilar una habitación privada o compartida penaliza el precio cayendo entre un 12% y un 52%.

Podemos observar en la imagen que el modelo arroja un coeficiente negativo para el aire acondicionado

(además de ser estadísticamente no significativo, como vimos en el summary anterior). Esto es debido a que en Sevilla el aire acondicionado está presente en casi el 95% de los alojamientos por lo que su presencia ya no sube el precio (es un requisito básico), pero los pisos que explícitamente lo anuncian como su gran extra suelen ser de gama media-baja intentando destacar, mientras que los pisos de gama alta esto lo dan por hecho.

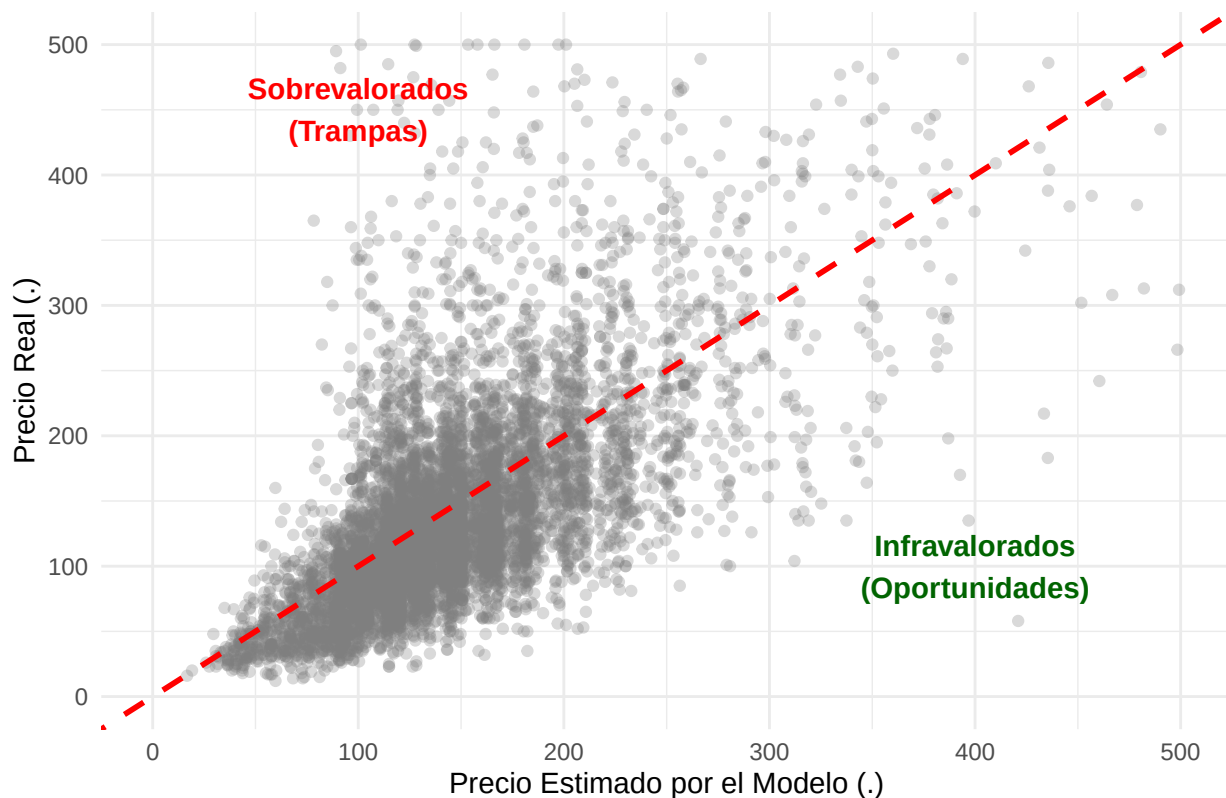
4.2.4 Detección de anomalías

```
# Calculamos predicciones y residuos
listings_4_2$precio_justo <- predict(modelo_precio, listings_4_2, type = "response")
listings_4_2$sobreprecio <- listings_4_2$price - listings_4_2$precio_justo

# A. Gráfico Radar (Scatter Plot)
ggplot(listings_4_2, aes(x = precio_justo, y = price)) +
  geom_point(alpha = 0.3, color = "gray50") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed", size = 1) +
  annotate("text", x = 100, y = 450, label = "Sobrevalorados\n(Trampas)", color = "red", fontface="bold") +
  annotate("text", x = 400, y = 100, label = "Infravalorados\n(Oportunidades)", color = "darkgreen", fontface="bold") +
  theme_minimal() +
  labs(title = "Radar de Mercado: Precio Real vs. Precio Justo",
       x = "Precio Estimado por el Modelo (€)", y = "Precio Real (€)") +
  xlim(0, 500) + ylim(0, 500)
```

```
## Warning: Removed 129 rows containing missing values or values outside the scale range (`geom_point`)
```

Radar de Mercado: Precio Real vs. Precio Justo



```
# B. Tablas de Casos Extremos
top_inflados <- listings_4_2 %>%
```

```

arrange(desc(sobreprecio)) %>%
select(id, neighbourhood_cleansed, price, precio_justo, sobreprecio) %>%
head(10)

top_chollos <- listings_4_2 %>%
  arrange(sobreprecio) %>%
  select(id, neighbourhood_cleansed, price, precio_justo, sobreprecio) %>%
  head(10)

print("--- Top 10 Trampas Turísticas ---")

```

```
## [1] "--- Top 10 Trampas Turísticas ---"
```

```
print(top_inflados)
```

```
##           id neighbourhood_cleansed price precio_justo sobreprecio
## 1 8.766069e+17          Feria      950    130.75173    819.2483
## 2 1.189635e+18          Feria      900     92.14655    807.8534
## 3 9.041080e+17          Feria      900    136.97825    763.0218
## 4 1.114446e+18      an Roque      900    168.92329    731.0767
## 5 8.819019e+06    San Bartolomé      900    180.52660    719.4734
## 6 1.491067e+18      Alfalfa      853    134.64251    718.3575
## 7 1.478324e+18 Encarnación, Regina      841    140.83464    700.1654
## 8 8.273759e+17 Encarnación, Regina      800    100.21224    699.7878
## 9 6.765653e+17      Alfalfa      800    105.91436    694.0856
## 10 6.686400e+17      Arenal      800    119.46386    680.5361

```

```
print("--- Top 10 Oportunidades ---")
```

```
## [1] "--- Top 10 Oportunidades ---"
```

```
print(top_chollos)
```

```
##           id neighbourhood_cleansed price precio_justo sobreprecio
## 1 4.783315e+06          Museo      263     890.1262   -627.1262
## 2 1.145272e+07      Alfalfa      350     920.9534   -570.9534
## 3 5.614541e+17      Alfalfa      436     952.1152   -516.1152
## 4 8.593089e+17      Arenal      605    1111.8851   -506.8851
## 5 1.145681e+18    La Florida      56     560.5523   -504.5523
## 6 1.141314e+18    Santa Cruz     993    1394.2240   -401.2240
## 7 1.207232e+18 Huerta de la Salud     378     757.9256   -379.9256
## 8 1.135258e+18    San Bartolomé      58     420.9644   -362.9644
## 9 1.767395e+07    San Bartolomé     549     903.9702   -354.9702
## 10 3.443300e+07    San Bartolomé     412     726.5464   -314.5464

```

En la gráfica anterior tenemos la proyección del “Precio Justo” contra el “Precio Real”, la línea diagonal roja representa el equilibrio del mercado perfecto.

Los puntos que nos encontramos muy por encima de la diagonal son alojamientos cuyo precio real excede lo que sus características objetivas justifican. Se pueden ver casos extremos en el barrio de Feria, donde pisos que deberían costar 127€ se ofertan a 950€. Esto es propio de especulación agresiva o eventos puntuales donde aprovechan la demanda turística.

Por debajo de la línea nos encontramos propiedades en La Florida y San Bartolomé con precios reales de 50 o 60 euros cuando su valoración matemática debería rondar los 300 o 400 euros. Esto puede deberse a errores en el precio por parte del anfitrión o estrategias de mercado para ganar las primeras valoraciones.

4.2.6 Modelo avanzado con interacciones

```
modelo_interaccion <- glm(price ~ neighbourhood_cleansed +
  accommodates +
  review_scores_rating +
  has_pool * room_type, # Interacción Clave
  family = Gamma(link = "log"),
  data = listings_4_2)

print("--- Resultados del Modelo con Interacción (GLM) ---")
```

```
## [1] "--- Resultados del Modelo con Interacción (GLM) ---"
```

```
coef_interaccion <- summary(modelo_interaccion)$coefficients
coef_interaccion[grep("pool", row.names(coef_interaccion)), ]
```

```
##              Estimate Std. Error   t value    Pr(>|t|)
## has_pool          0.3274110 0.02443861 13.397286 1.859326e-40
## has_pool:room_typeHotel room -0.5933260 0.28858953 -2.055951 3.982236e-02
## has_pool:room_typePrivate room 0.2468972 0.08050459  3.066871 2.170956e-03
```

En este apartado planteamos la hipótesis de que el valor de un servicio como es la piscina no es constante, sino que depende de la tipología del alojamiento, es decir, ¿paga el cliente lo mismo por una piscina privada que por una comunitaria cuando alquila una habitación?

Para ello hemos introducido en el modelo un término de interacción **has_pool x room_type**. Como vemos en los resultados, el coeficiente base de contar con piscina (has_pool) arroja un valor de 0.327. Esto significa que, para la categoría de referencia (el apartamento completo estándar), la piscina es un diferenciador absoluto que eleva el precio base de la propiedad en un 38,7% ($\exp(0.327)-1$).

Si observamos los coeficientes de interacción, el modelo revela dinámicas de mercado radicalmente distintas según el tipo de producto:

Por un lado, encontramos una clara anomalía en las habitaciones de hotel, donde vemos un coeficiente de interacción fuertemente negativo (-0.593) y estadísticamente significativo ($p < 0.05$). Esto indica que el modelo penaliza severamente esta combinación. Al sumar el efecto base y la interacción, descubrimos que los hoteles con piscina son porcentualmente más baratos que los que no la tienen. Esto confirma que la categoría hotelera en Airbnb agrupa dos realidades opuestas: los “Hoteles Boutique” del centro histórico (carísimos, pero sin espacio físico para piscina) y cadenas hoteleras u hostales de la periferia (con piscina, pero con precios base mucho más bajos), rompiendo la lógica residencial del dataset.

Por otro lado, el hallazgo más revelador se encuentra en la habitación privada (Private room). Su coeficiente de interacción es netamente positivo ($+0.246$) y altamente significativo ($p < 0.01$). Si sumamos este efecto al valor base de la piscina, descubrimos que una piscina en una habitación privada eleva su cotización en un 77,5% ($\exp(0.327+0.246)-1$). Esto demuestra que alquilar una habitación con piscina cambia por completo la gama del producto, pasando de un alojamiento puramente económico a una experiencia de lujo compartido (ej. chalets residenciales), justificando un salto porcentual en el precio mucho mayor que en un piso completo.

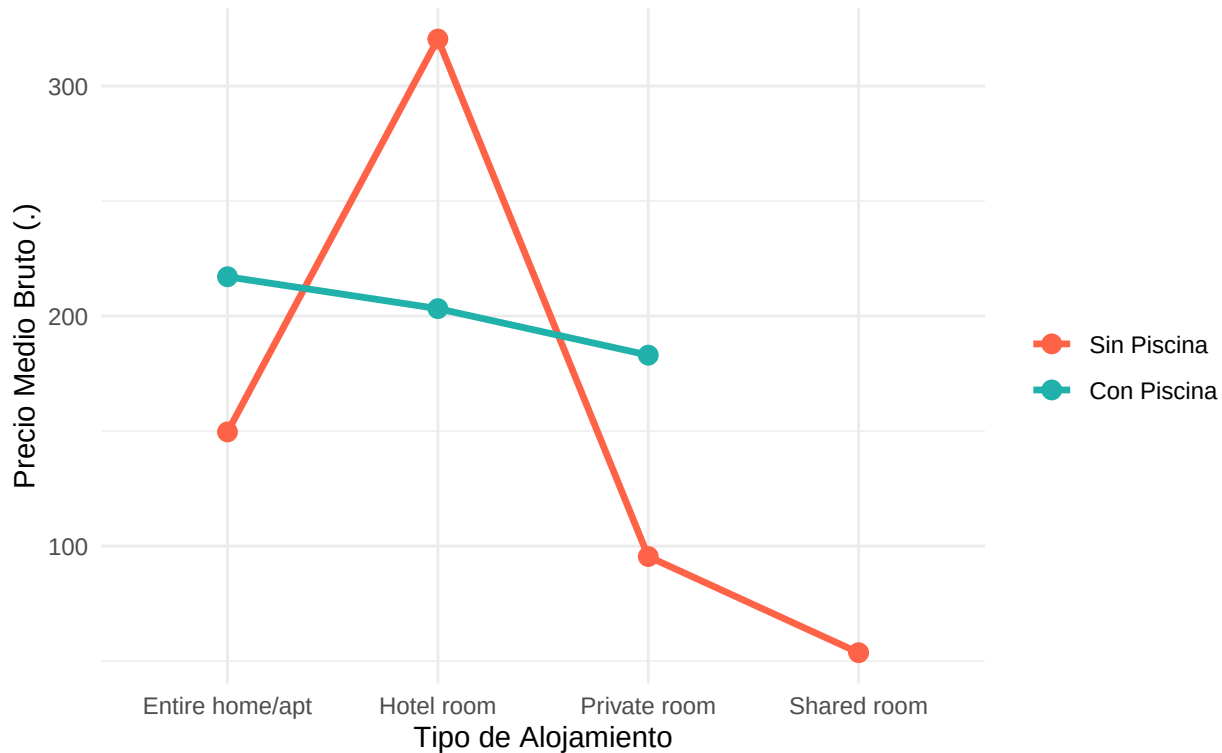
Como conclusión final, podemos afirmar que la regla de oro de la revalorización por piscina es robusta en el mercado residencial (disparando el precio de pisos y habitaciones), pero sufre un efecto inverso en la oferta hotelera listada en la plataforma, la cual obedece a dinámicas patrimoniales de ubicación en el casco antiguo.

```
# Calculamos el precio medio según si tiene piscina y el tipo de alojamiento
interaccion_plot <- listings_4_2 %>%
  group_by(room_type, has_pool) %>%
  summarise(precio_medio = mean(price), .groups = 'drop') %>%
  mutate(has_pool = factor(has_pool, labels = c("Sin Piscina", "Con Piscina")))
```

```
ggplot(interaccion_plot, aes(x = room_type, y = precio_medio, group = has_pool, color = has_pool)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  theme_minimal() +
  scale_color_manual(values = c("tomato", "lightseagreen")) + # Colores coherentes
  labs(title = "¿Vale la pena poner piscina?",
       subtitle = "La revalorización depende drásticamente de la tipología del activo",
       x = "Tipo de Alojamiento", y = "Precio Medio Bruto (€)", color = "")
```

¿Vale la pena poner piscina?

La revalorización depende drásticamente de la tipología del activo



Hemos generado un gráfico en el que podemos observar cómo varía el precio medio según si aparece piscina en cada tipo de alojamiento.

Observamos una clara correlación positiva en los segmentos residenciales, en ambos casos la línea azul se sitúa por encima de la roja. En cambio, como hemos visto antes en los hoteles, las líneas se cruzan y aparecen hoteles sin piscina con precios mucho más altos que los que tienen piscina.

4.2.7 Calculadora de precio justo

Para finalizar el análisis, hemos transformado el modelo en una herramienta de tasación. Esta herramienta permite a los usuarios introducir las características de su alojamiento y obtener una estimación de valor basada en los 7300 registros que componen nuestro modelo.

A continuación, se presenta un caso de uso: - Ubicación: Triana Oeste - Tipología: Piso completo (Entire home/apt) - Capacidad: 4 personas - Extras: Piscina y aire acondicionado - Calidad Esperada: Calificación de 4.8/5

```
predecir_precio <- function(barrio, tipo, capacidad, baños, piscina, ac, rating) {
  nuevo_piso <- data.frame(
    neighbourhood_cleansed = barrio,
```

```

room_type = tipo,
accommodates = capacidad,
bathrooms = baños,
has_pool = piscina, # 1 o 0
has_ac = ac, # 1 o 0
review_scores_rating = rating
)
prediccion <- predict(modelo_precio, newdata = nuevo_piso, type = "response")

return(paste0("El precio justo de mercado para este piso es: ", round(prediccion, 2), "€"))
}
# "¿Cuánto debería cobrar por un piso en Triana para 4 personas con piscina?"
print(prededir_precio("Triana Oeste", "Entire home/apt", 4, 1, 1, 1, 4.8))

```

```
## [1] "El precio justo de mercado para este piso es: 169.52€"
```

El algoritmo, aplicando los coeficientes porcentuales del GLM, nos arroja un valor técnico de mercado en torno a los 169€ la noche.

Este valor funciona como precio base. A partir de aquí, el propietario puede tomar decisiones basadas en datos: si decide cobrar 250€, sabrá a ciencia cierta que está exigiendo un sobreprecio de casi el 48% respecto a la media de sus competidores directos, un sobrecoste que deberá justificar con intangibles (interiorismo de lujo, vistas, marca personal). Por el contrario, si decide salir a comercializarlo a 130€, tendrá la seguridad matemática de estar siendo agresivo en precio y estar situado en la zona de oportunidad del mercado, lo que muy probablemente le garantizará llenar el calendario de reservas rápidamente en sus primeros meses.

4.3. El “Efecto Giralda”

Para abordar la hipótesis del ‘Efecto Giralda’, comenzamos analizando la distribución espacial de los precios como un continuo geográfico. Al prescindir de las fronteras administrativas de los barrios (que a menudo son arbitrarias y no reflejan la realidad del mercado turístico), buscamos observar si existe un patrón radial de encarecimiento que tenga como epicentro absoluto el principal monumento de la ciudad: la Catedral/Giralda.

Dado que el dataset original solo contenía coordenadas geográficas (latitud/longitud), era imprescindible transformarlas en una métrica de negocio utilizable.

Para ello, hemos aplicado la fórmula de Haversine, calculando la distancia de círculo máximo (en metros) entre cada alojamiento y la Catedral, teniendo en cuenta la curvatura de la Tierra. Gracias a esta nueva variable (distancia_giralda_m), hemos podido acotar el análisis a un radio operativo de 3.000 metros y generar la siguiente visualización de “Calor Radial”:

```

# --- Visualización "Diana" del Efecto Giralda ---

library(dplyr)
library(ggplot2)
library(geosphere) # Necesario para calcular la distancia

# 1. Definimos el centro (Giralda)
centro_giralda <- c(-5.99238, 37.38614)

# 2. CALCULAMOS LA DISTANCIA (El paso que faltaba antes)

df_eda <- df_eda %>%
  rowwise() %>%
  mutate(
    distancia_giralda_m = distHaversine(c(longitude, latitude), centro_giralda)
  )

```



```

) %>%
ungroup()

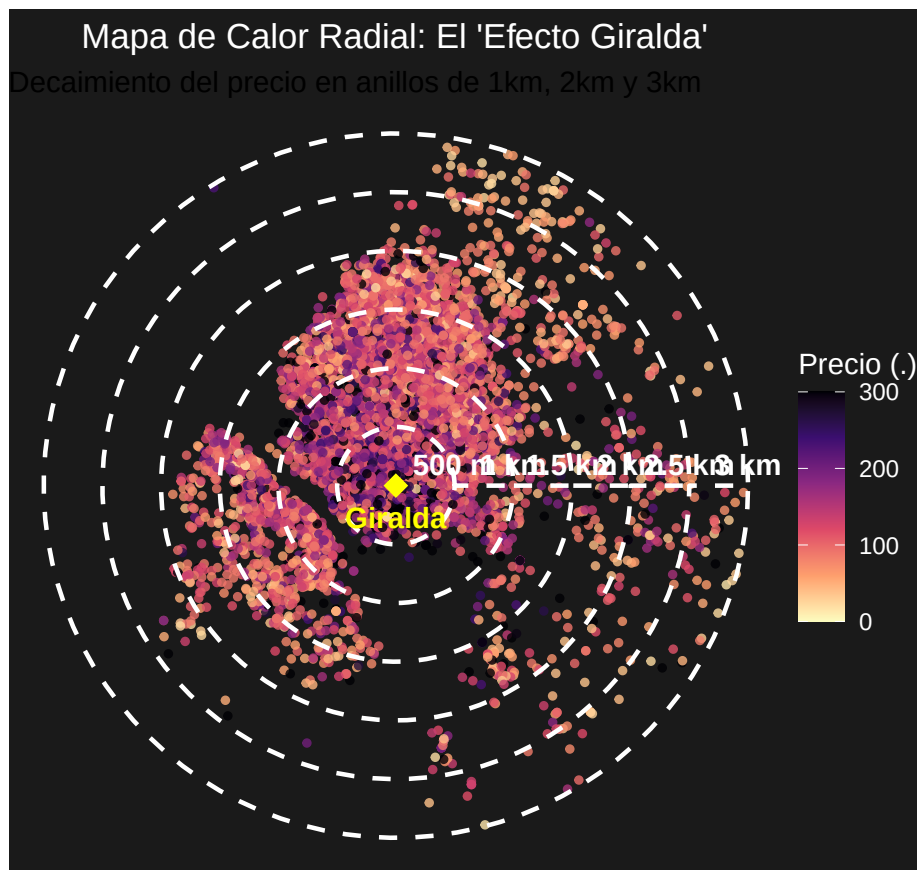
# 3. Ahora sí, creamos df_diana filtrando por esa distancia
df_diana <- df_eda %>%
  filter(distancia_giralda_m <= 3000)

# 4. Generamos los anillos de la diana (1km, 2km, 3km)
crear_anillo <- function(radio_metros, etiqueta) {
  angulos <- seq(0, 2*pi, length.out = 100)
  r_lat <- radio_metros / 111132.95
  r_lon <- radio_metros / (111132.95 * cos(centro_giralda[2] * pi/180))
  data.frame(
    x = centro_giralda[1] + r_lon * cos(angulos),
    y = centro_giralda[2] + r_lat * sin(angulos),
    label = etiqueta
  )
}

anillos <- bind_rows(
  crear_anillo(500, "500 m"),
  crear_anillo(1000, "1 km"),
  crear_anillo(1500, "1.5 km"),
  crear_anillo(2000, "2 km"),
  crear_anillo(2500, "2.5 km"),
  crear_anillo(3000, "3 km")
)

# 5. Pintamos el Mapa
ggplot() +
  geom_point(data = df_diana, aes(x = longitude, y = latitude, color = price),
    size = 1, alpha = 0.8) +
  geom_path(data = anillos, aes(x = x, y = y), color = "white", linetype = "dashed", size = 0.8) +
  geom_text(data = anillos %>% group_by(label) %>% slice(1),
    aes(x = x, y = y, label = label), color = "white", vjust = -0.5, fontface = "bold") +
  annotate("point", x = centro_giralda[1], y = centro_giralda[2], color = "yellow", size = 4, shape = 1) +
  annotate("text", x = centro_giralda[1], y = centro_giralda[2], label = "Giralda",
    vjust = 2, color = "yellow", fontface = "bold", size = 4) +
  scale_color_viridis_c(option = "magma", direction = -1, limits = c(0, 300), oob = scales::squish) +
  coord_quickmap() +
  theme_void() +
  theme(plot.background = element_rect(fill = "gray10", color = NA),
    legend.text = element_text(color = "white"),
    legend.title = element_text(color = "white"),
    plot.title = element_text(color = "white", hjust = 0.5)) +
  labs(title = "Mapa de Calor Radial: El 'Efecto Giralda'",
    subtitle = "Decaimiento del precio en anillos de 1km, 2km y 3km",
    color = "Precio (€)")

```



Como podemos observar en la visualización, los puntos más oscuros (precios más altos) se agrupan formando una densa nube alrededor del marcador amarillo de la Giralda. A medida que nuestra vista se desplaza hacia la periferia, los colores viran hacia tonos más claros, sugiriendo una caída del precio. Sin embargo, esta intuición visual no es suficiente para extraer conclusiones de negocio; necesitamos cuantificar matemáticamente esta pérdida de valor.

Una vez calculada la distancia exacta a la Giralda, procedemos a modelar el decaimiento del valor económico. Para evitar el sesgo introducido por los diferentes tipos de alojamiento, aislaremos el análisis exclusivamente a la categoría 'Entire home/apt' y limitaremos el radio de estudio a los primeros 3.000 metros (área de influencia turística). Dado que la caída de precios en bienes raíces rara vez es perfectamente lineal (el precio tiende a estabilizarse en la periferia), utilizaremos una técnica de Regresión No Lineal (LOESS) para capturar la curvatura real del mercado, complementada con un modelo lineal clásico para los primeros 1.000 metros y así responder con precisión a nuestra pregunta de negocio.

```
# 1. Filtramos datos para comparar "manzanas con manzanas"
df_giralda <- df_eda %>%
  filter(
    room_type == "Entire home/apt", # Solo pisos completos
    price < 400,                    # Filtramos lujo extremo para ver la tendencia general
    distancia_giralda_m <= 3000    # Analizamos solo los primeros 3 kilómetros
  )

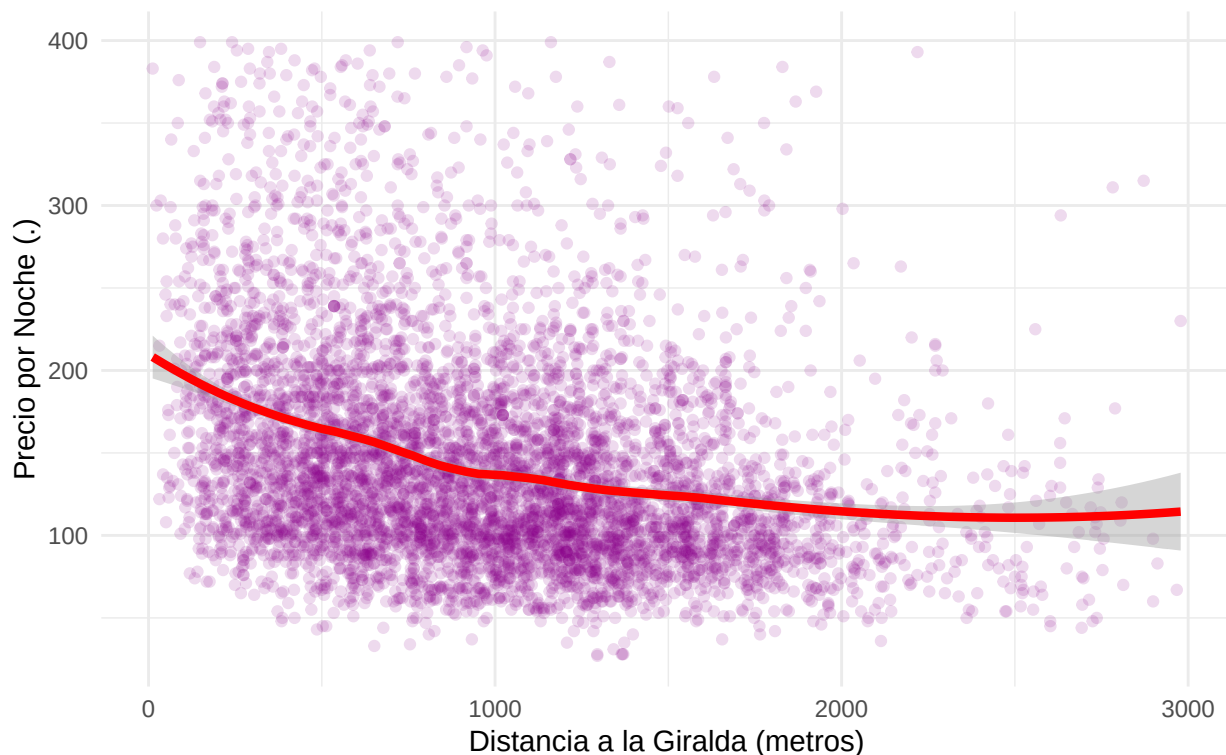
# 2. Visualización del Modelo LOESS
ggplot(df_giralda, aes(x = distancia_giralda_m, y = price)) +
  geom_point(alpha = 0.15, color = "darkmagenta") +
  # Añadimos la curva LOESS para ver la tendencia no lineal
  geom_smooth(method = "loess", color = "red", linewidth = 1.5, span = 0.5) +
```

```
theme_minimal() +
labs(
  title = "El Efecto Giralda: Decaimiento del Precio Turístico",
  subtitle = "La curva roja (LOESS) muestra cómo el precio se desploma en el primer kilómetro y luego",
  x = "Distancia a la Giralda (metros)",
  y = "Precio por Noche (€)"
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

El Efecto Giralda: Decaimiento del Precio Turístico

La curva roja (LOESS) muestra cómo el precio se desploma en el primer kilómetro y luego



```
# 3. Modelo Matemático (Regresión Lineal) para el primer kilómetro
# Queremos saber el dato exacto de cuánto cae por cada 100 metros en la zona más sensible
df_primer_km <- df_giralda %>% filter(distancia_giralda_m <= 1000)
modelo_lineal <- lm(price ~ distancia_giralda_m, data = df_primer_km)

# Extraemos el coeficiente (cuánto cae el precio por cada 1 metro) y lo multiplicamos por 100
caida_por_metro <- coef(modelo_lineal)["distancia_giralda_m"]
caida_por_100m <- caida_por_metro * 100

print(paste("Caída del precio por cada 100 metros en el primer kilómetro:", round(caida_por_100m, 2), "
```

```
## [1] "Caída del precio por cada 100 metros en el primer kilómetro: -6.5 €"
```

Respondiendo a la pregunta planteada: Sí, el 'Efecto Giralda' es un fenómeno cuantificable en el mercado de Airbnb en Sevilla. A través de la regresión local (LOESS), hemos descubierto que el comportamiento espacial del precio no es constante, sino que presenta dos fases diferenciadas:

1.(0 - 1.000 metros): En el radio inmediato a la Catedral, la penalización por alejarse es severa. Nuestro

modelo de regresión lineal aplicado a este segmento revela que el precio del alquiler de un apartamento completo cae, en promedio, 6,50 € por cada 100 metros que nos alejamos del epicentro. Esto demuestra que en el centro histórico, la ubicación extrema prima sobre el resto de variables.

2.(> 1.500 metros): A partir del kilómetro y medio, la curva roja (LOESS) frena su caída y se estabiliza casi por completo en un suelo de precios base. En este punto, la variable ‘distancia a la Giralda’ pierde todo su poder predictivo frente a las características internas del inmueble (tamaño, piscina, etc).

Impacto de Negocio: El sobrecoste de adquirir una vivienda a menos de 500 metros de la Giralda está justificado por una prima de facturación diaria asegurada. Sin embargo, más allá de la frontera del primer kilómetro, la estrategia óptima para un inversor no es buscar cercanía al centro, sino maximizar el equipamiento (amenities) del piso.